人工智能与深度学习

基于深度学习的机器视觉

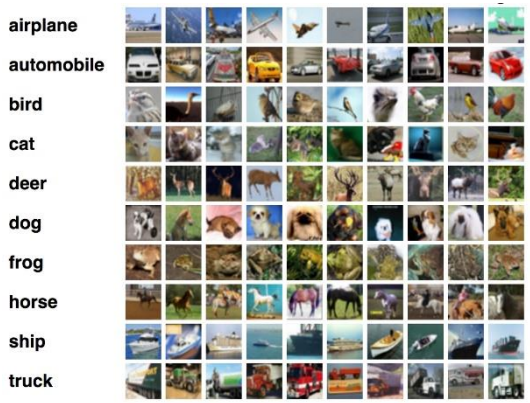
华中科技大学 机械科学与工程学院
陈伟

2026

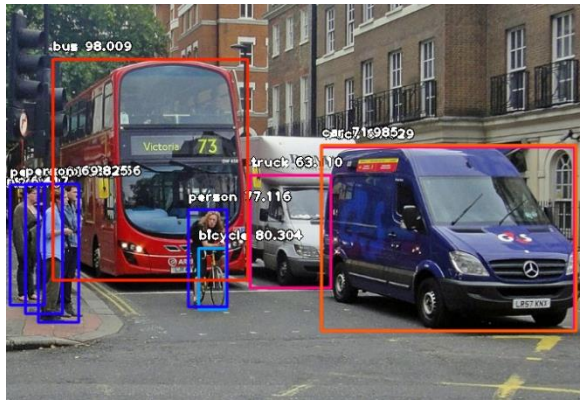
常见的机器视觉任务

常见的机器视觉（computer vision）任务

- 常见的机器视觉任务包括了图像的分类（classification）、检测（detection）和分割（segmentation）等



分类



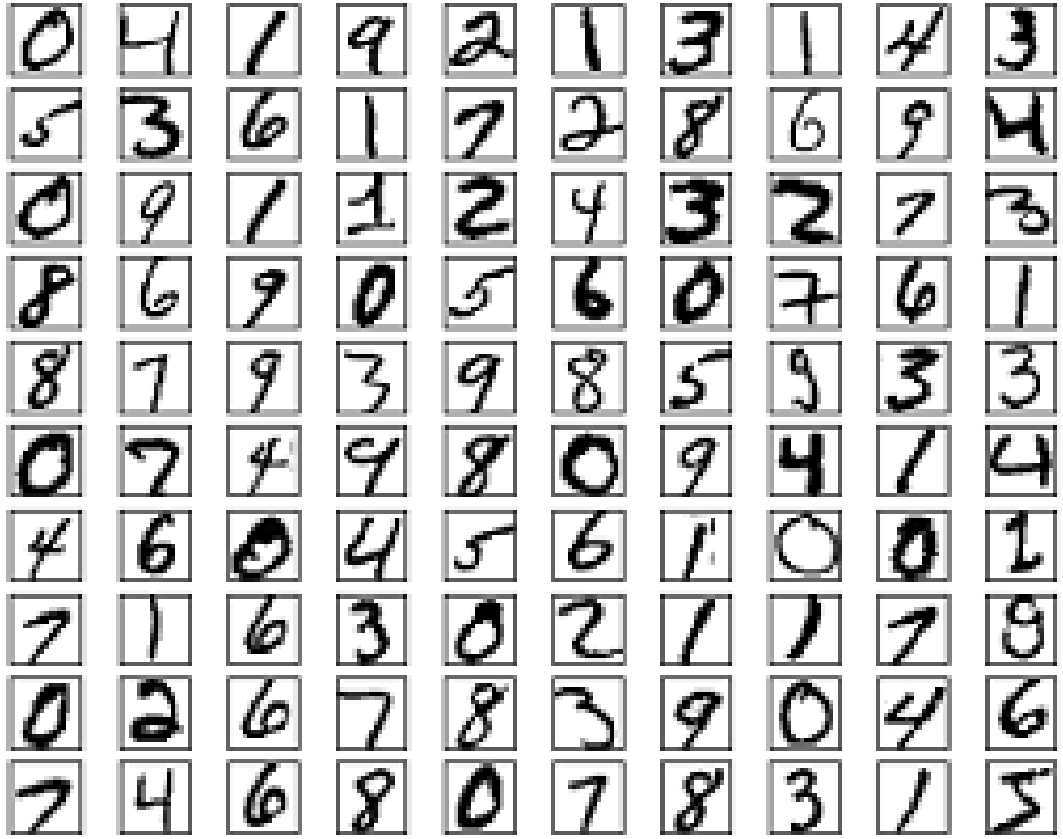
检测



分割

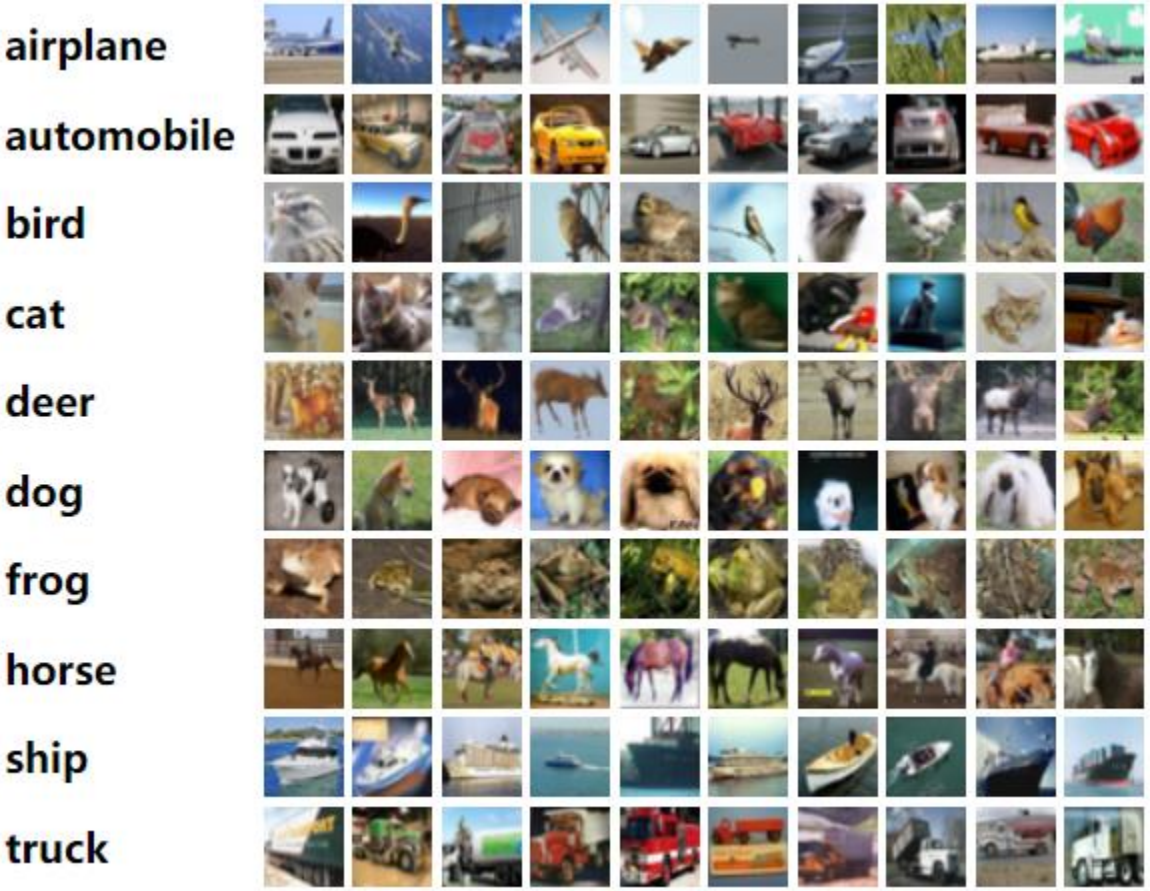
图像分类

- MNIST 手写数字



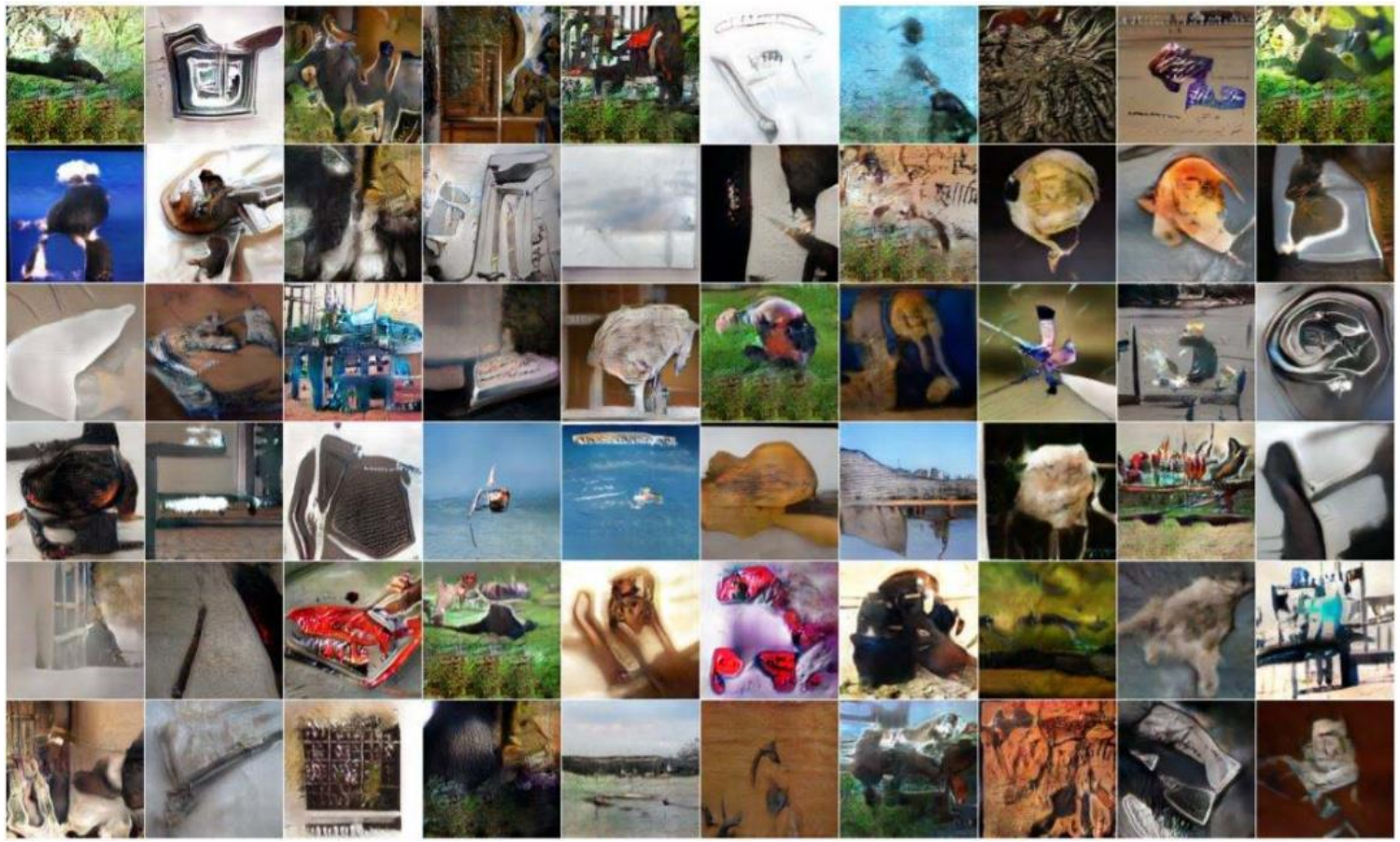
图像分类

- CIFAR-10 数据集：50000 训练图像、10000 测试图像、10个类别



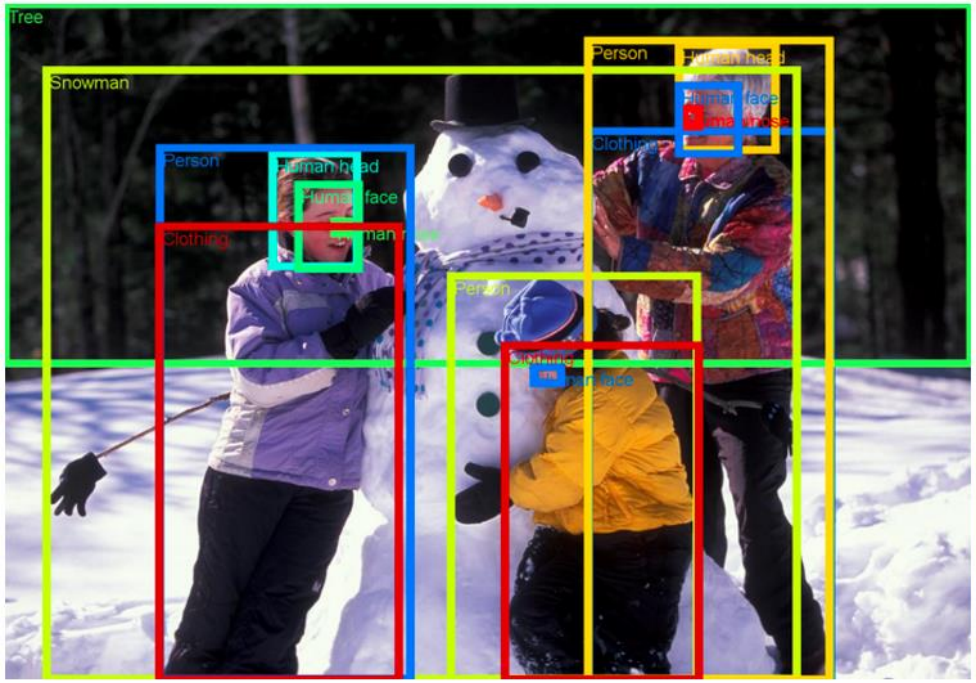
图像分类

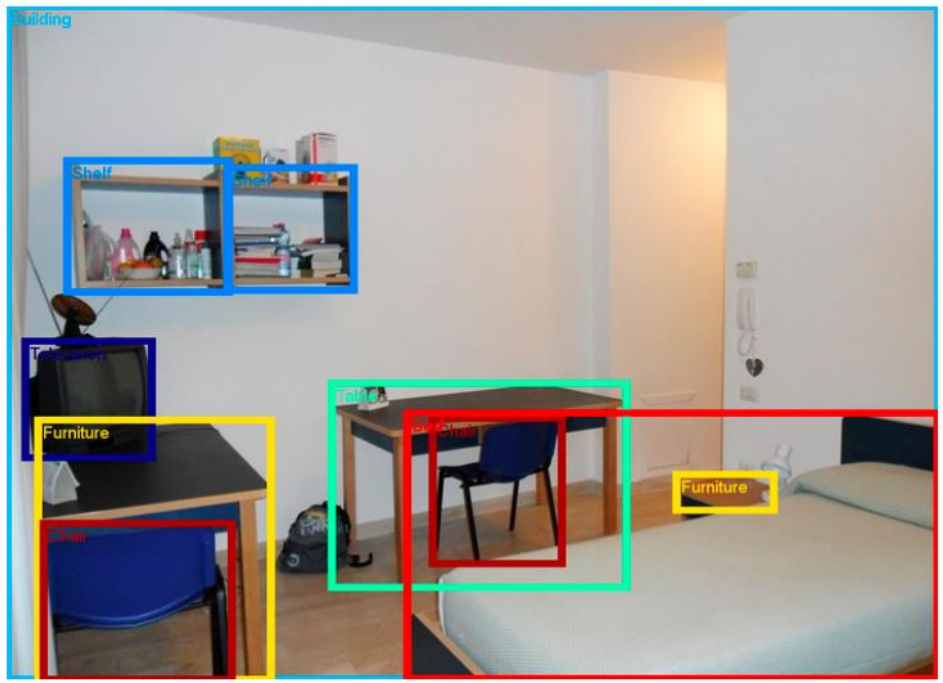
- ImageNet 数据集：1200000 训练图像，50000 验证图像，1000个类别



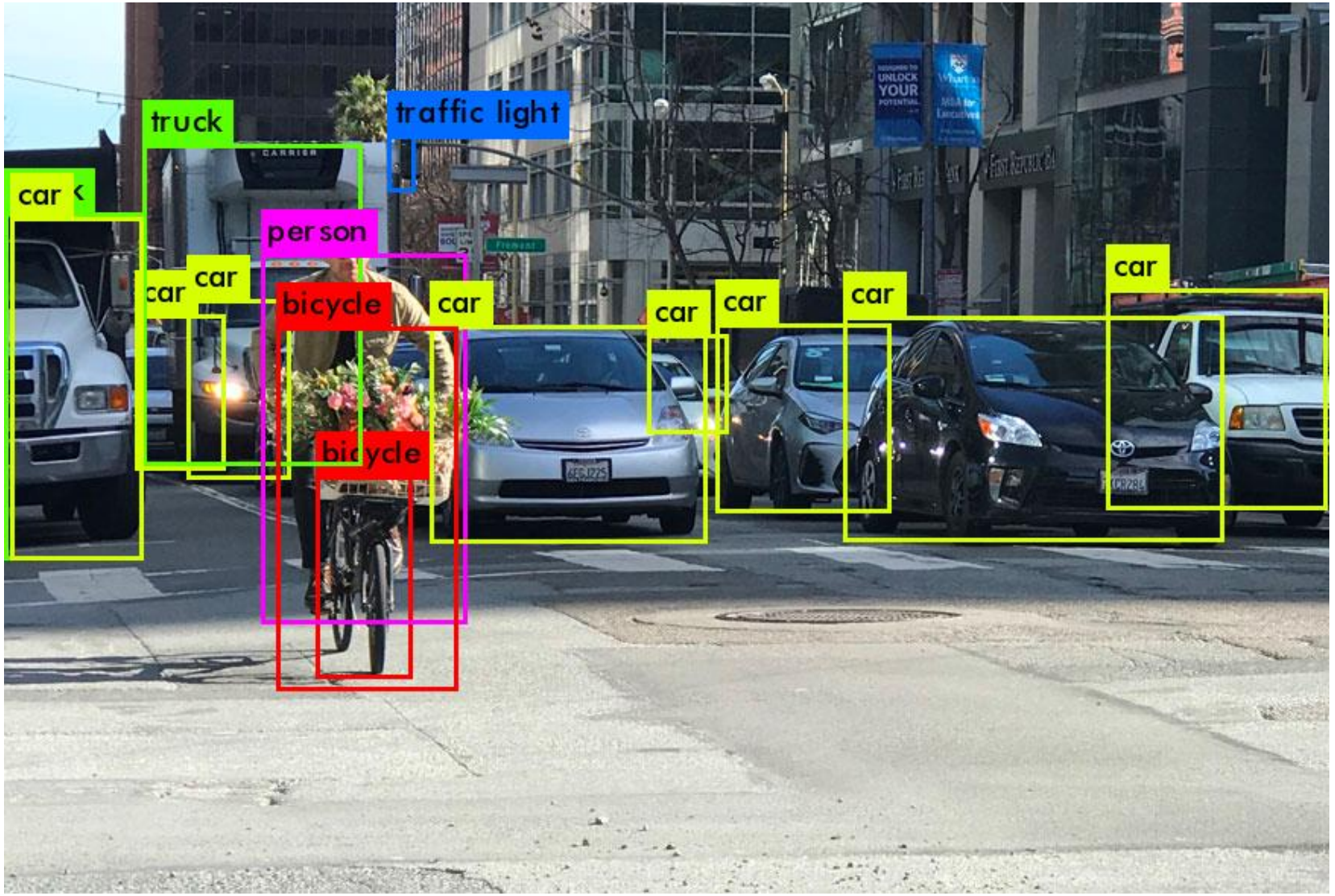
目标检测

- 这里提到的目标检测，包含了检测以及对检测对象的定位（即需要画出 bounding box）



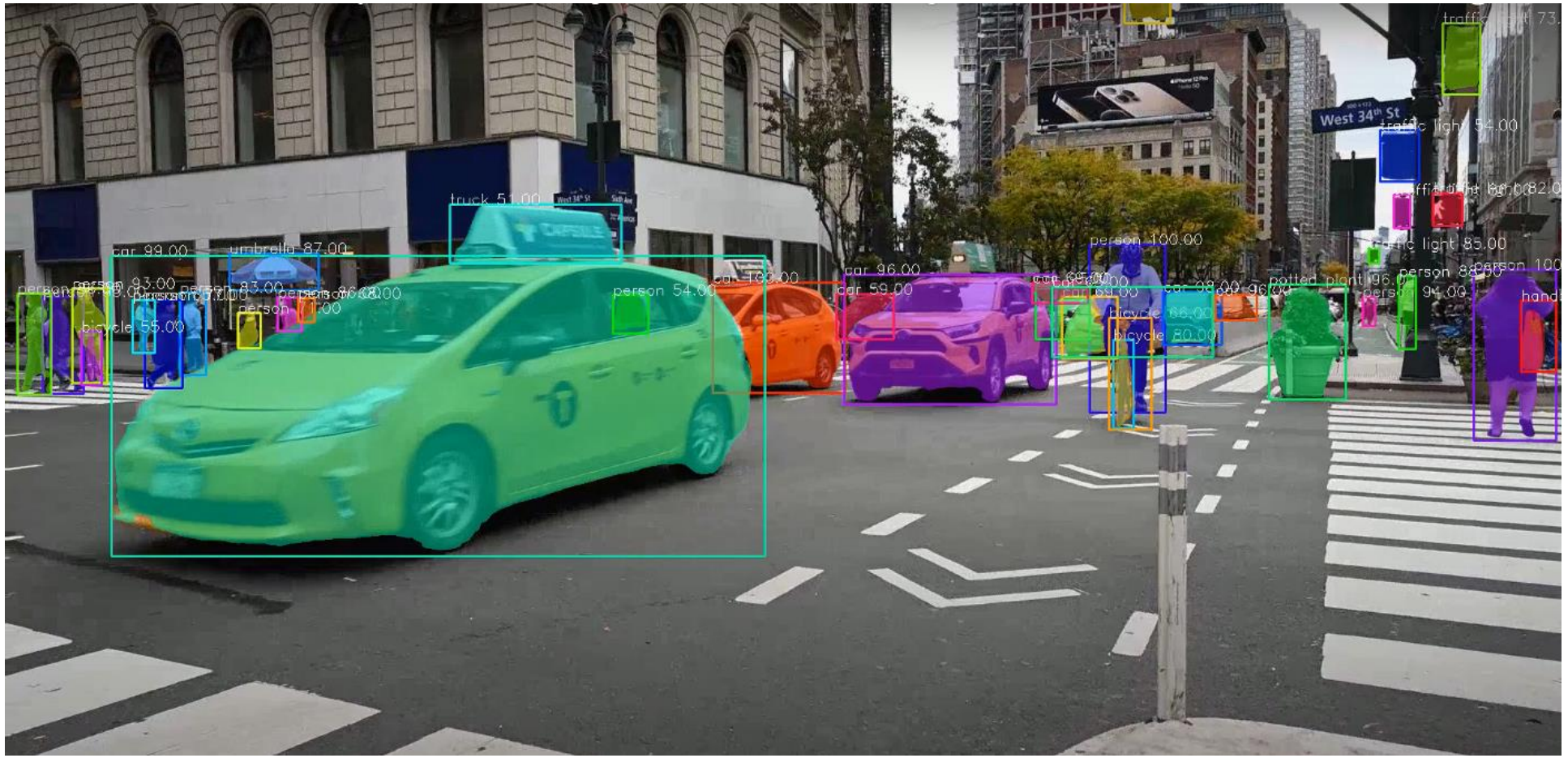


目标检测


truck
traffic light
car
person
car
car
bicycle
car
car
car
car
car
bicycle

目标分割

- 判断图像中哪些像素属于哪个目标



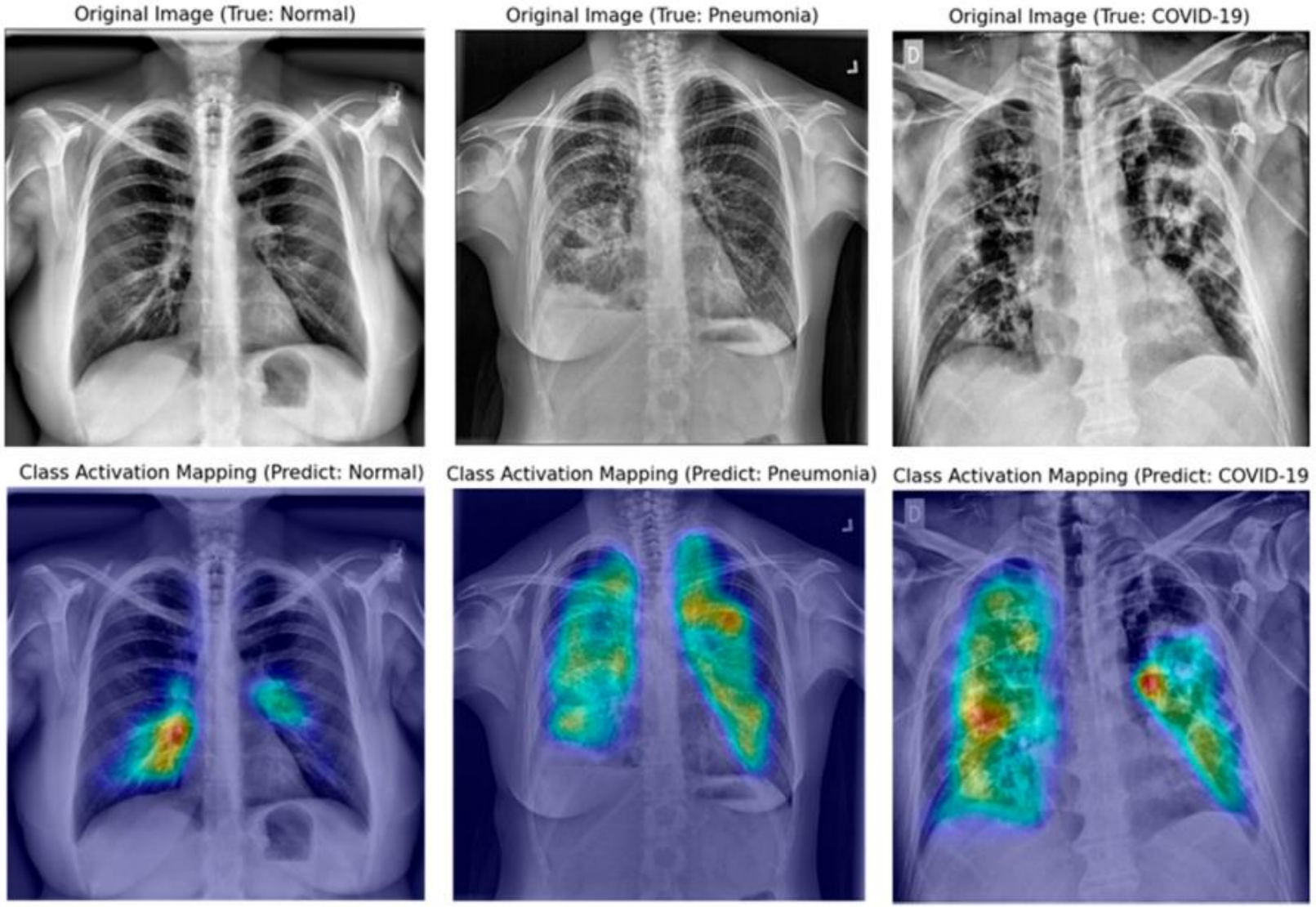
目标分割



更复杂的机器视觉任务

- 在分类、检测与分割技术的基础上，机器视觉还可以完成更加复杂任务

AI 医疗诊断

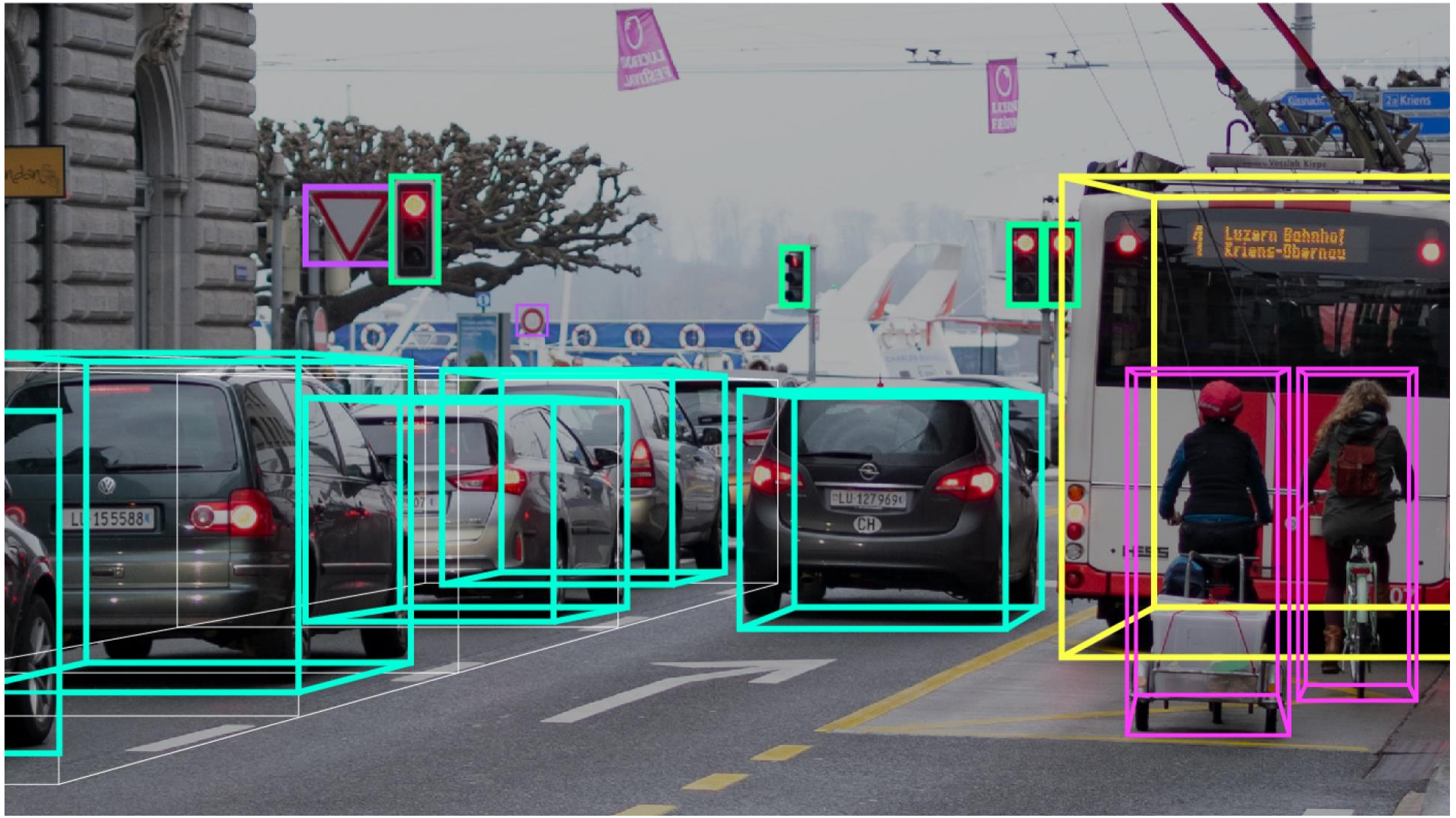

Original Image (True: Normal)
Original Image (True: Pneumonia)
Original Image (True: COVID-19)
Class Activation Mapping (Predict: Normal)
Class Activation Mapping (Predict: Pneumonia)
Class Activation Mapping (Predict: COVID-19

人体姿态等复杂信息的识别



三维目标检测

- 得到三维的 bounding box 信息



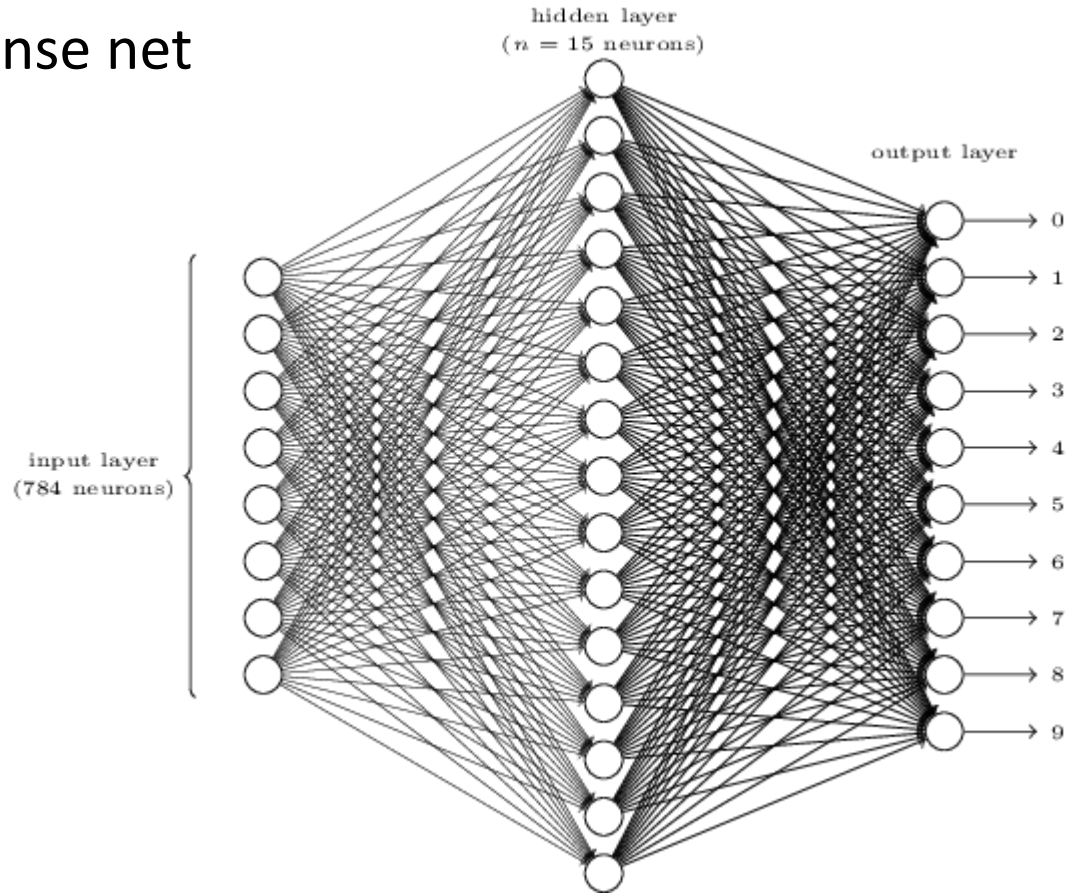
生成新图像



卷积神经网络
(Convolutional Neural Networks, CNN)

回顾：MNIST 手写数字识别

- **我们之前在处理28*28的灰度图片时，使用了一层全连接的网络（fully connected layer）**
  - 隐藏层的15个神经元，每个都与输入层784个神经元相连接
  - 全连接的网络有时也叫dense net



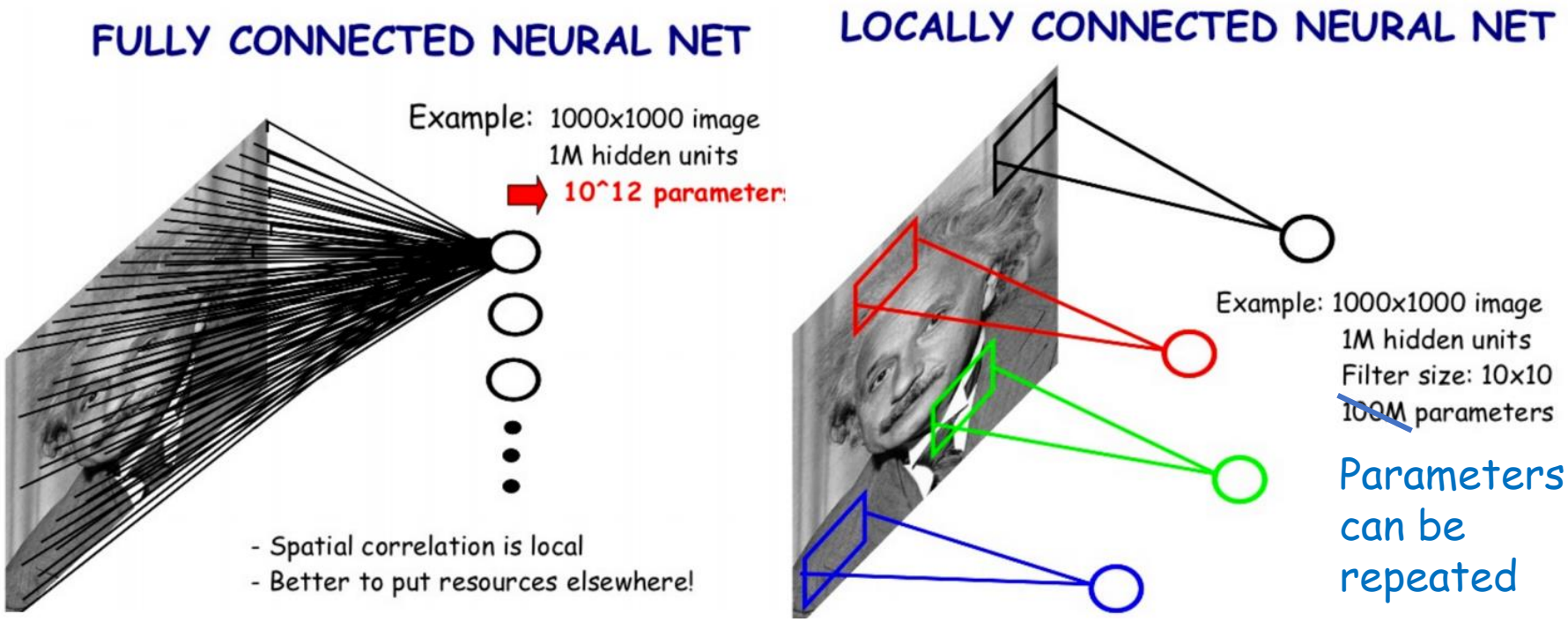
注：输入层有28x28=784个神经元，图中没有全部画出来

回顾：全连接网络

- **全连接网络需要训练大量参数不同的参数**
  - 参数数量：输入层神经元数目 * 输出层神经元数目
- **全连接网络没有充分利用图像数据的特性**
  - 将二维的图像展开成一维的向量，会丢失空间信息
  - 参数太多会导致训练困难，也会容易造成过拟合

全连接与局部连接

- 我们希望从图像局部信息开始，提取关键特征



卷积神经网络（CNN，ConvNet）

- **一个典型的卷积神经网络包含以下部分**
  - 卷积（convolutional）层
  - ReLU层
  - 池化（pooling）层
  - 全连接层、softmax层等

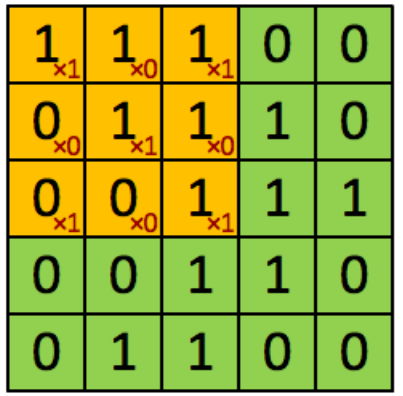
  - 注：有些说法中，ReLu层是包含在卷积层中的，即卷积运算+ReLu激活函数一起，构成卷积层

卷积层：过滤器（filter）

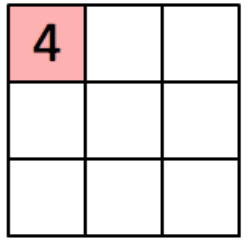
- **考虑一个5*5的黑白图像（每个像素要么是0，要么是1）**
- **卷积层的输入，就是这个5*5的二维数组**
- **卷积层中，使用一个3*3的过滤器**
  - 过滤器有时也被叫做核（kernel）
  - 过滤器的中参数（即3*3的矩阵），就是神经网络需要学习的参数

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

过滤器



Image



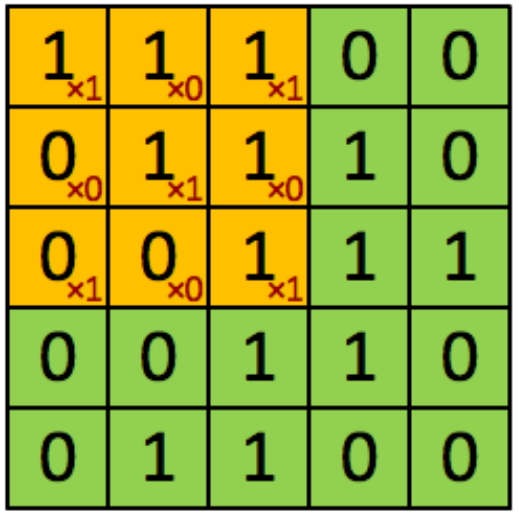
Convolved Feature

卷积层：过滤器

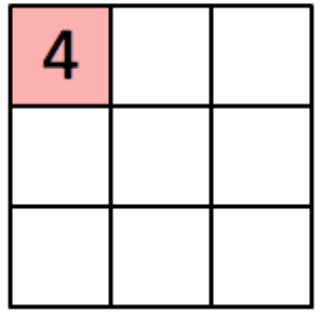
- **在给定过滤器参数的情况下，卷积层按照如下方式计算输出**

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

过滤器



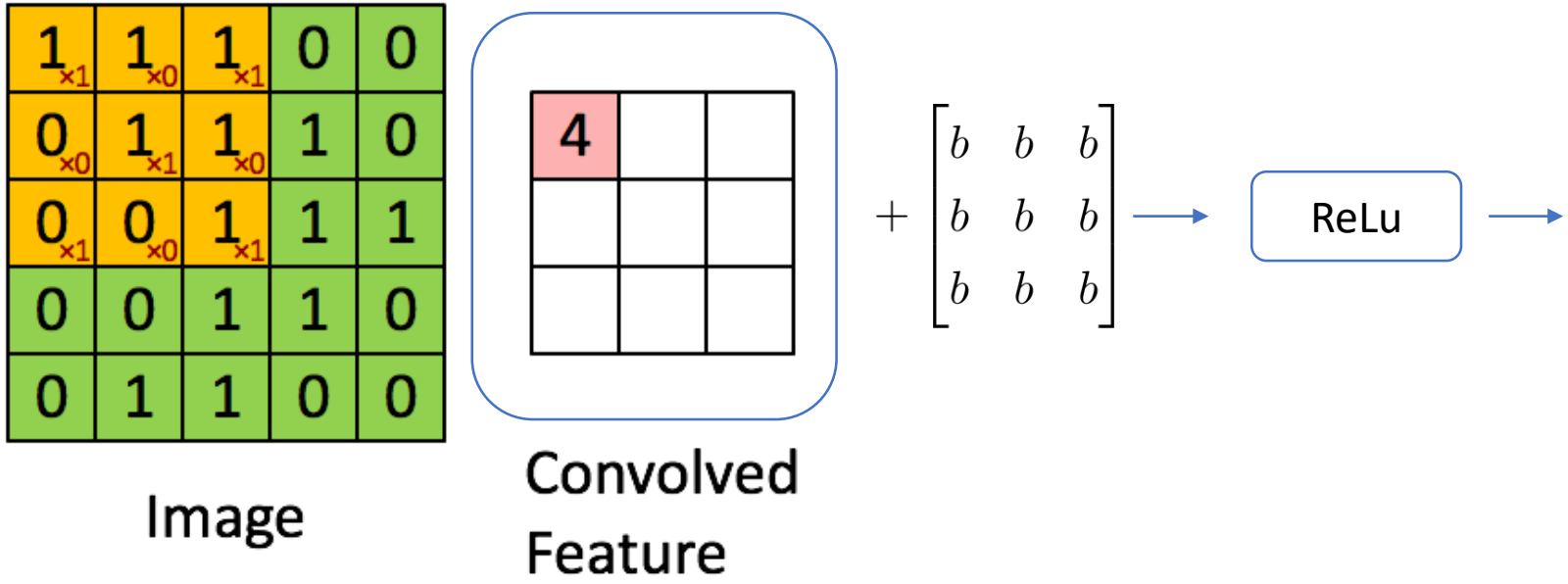
Image



Convolved Feature

- **此时的输出是3*3数组**
  - 假如输入图像是6*6，过滤器仍是3*3，则输出为4*4

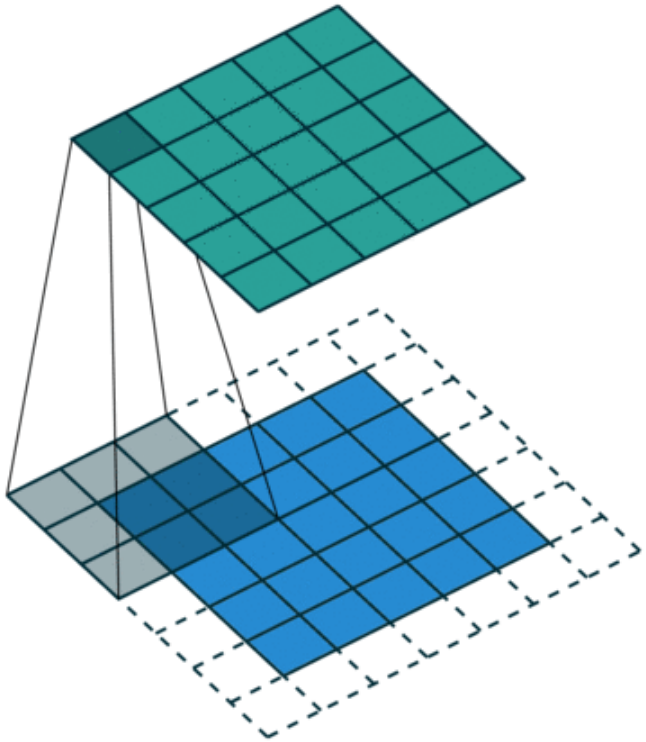
卷积层：过滤器与ReLu



- **对卷积之后的结果，我们可以加上偏置b，再通过ReLu激活函数，得到卷积层的输出**
  - 图中情况的输出仍是 3*3 矩阵

卷积层：填充（padding）

- **在上页的卷积层中，图像边缘的信息似乎利用得不充分**
- **我们可以对输入进行填充，以更好地利用边缘信息**
  - 可以填充一圈零值
  - 也可以根据需要填充多圈
  - 也可以不填充



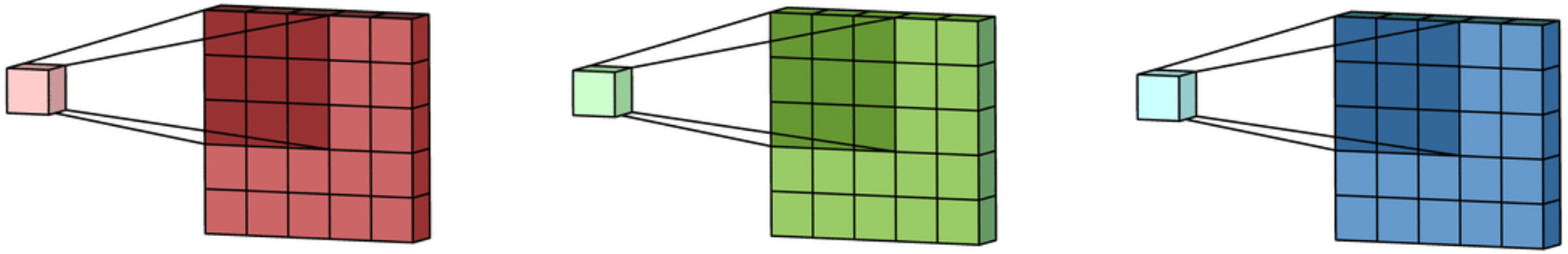
卷积层：多通道（multiple channels）

- **处理黑白图象时，输入的数据大小是长*宽**
- **彩色图像使用RGB三个通道描述颜色，此时输入层的大小为长*宽*3**
  - 这种三维或多维的数组，我们可以称之为张量（tensor）

- **对于多通道的图像，我们也可以使用卷积**
  - 对于5*5*3的图像，我们可以使用3*3*3的卷积
  - 首先对每个通道单独进行卷积，得到多个二维数组（即对三个不同的5*5的图像，分别使用三个不同的3*3过滤器）
  - 将每个通道通过卷积后的结果相加，再加上偏置

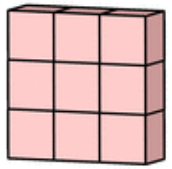
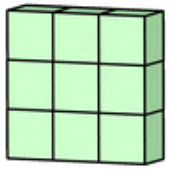
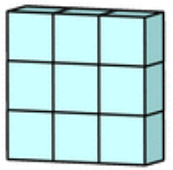
卷积层：多通道

- 分别卷积（使用不同的过滤器）



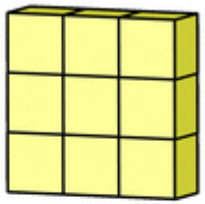
卷积层：多通道

- 卷积后三个通道相加

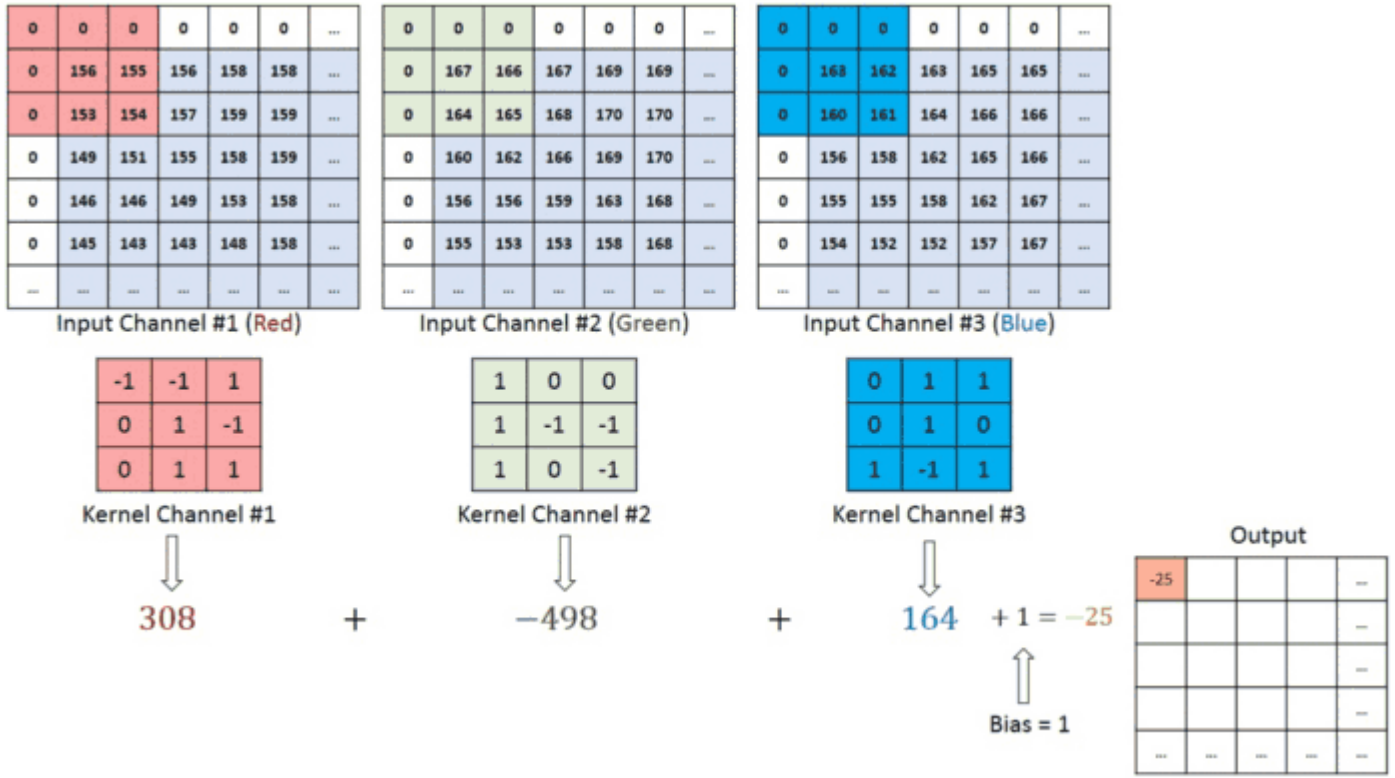




卷积层：多通道

- 再加入偏置

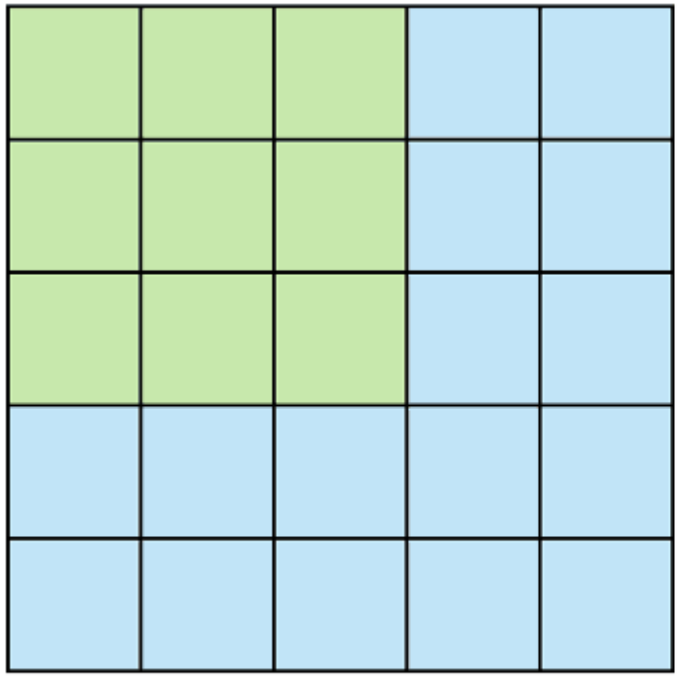


卷积层：多通道示例

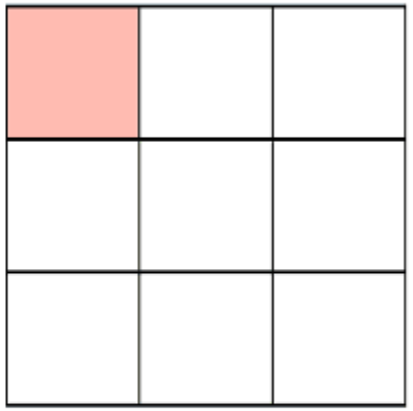

Input Channel #1 (Red)
Input Channel #2 (Green)
Input Channel #3 (Blue)
Kernel Channel #1
Kernel Channel #2
Kernel Channel #3
308
+
−498
+
164
+ 1 = −25
Bias = 1
Output

卷积层：步长（stride）

- 在卷积中，我们使用一个过滤器，在输入图像上“滑动”



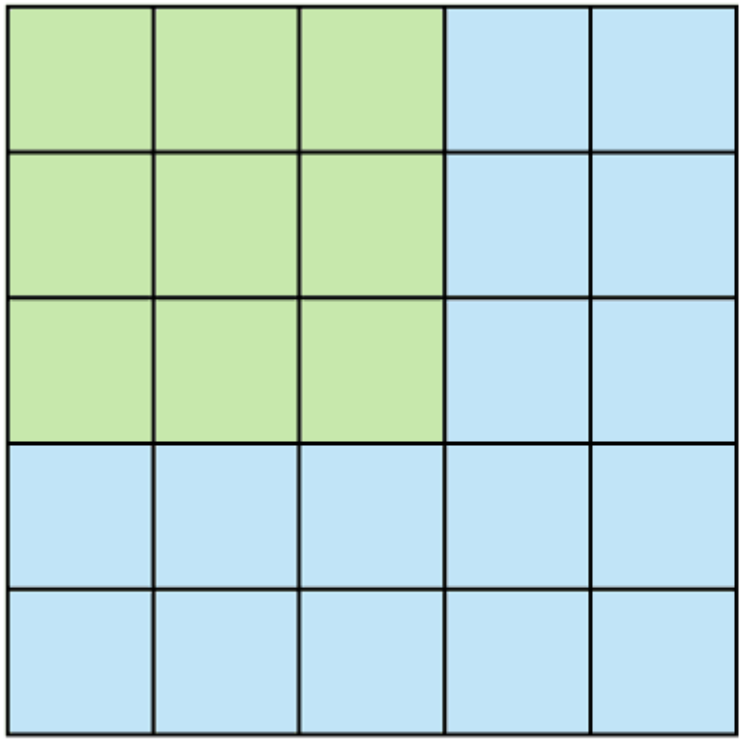
Stride 1



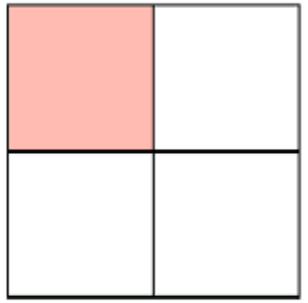
Feature Map

卷积层：步长

- “滑动”的步长，可以大于1



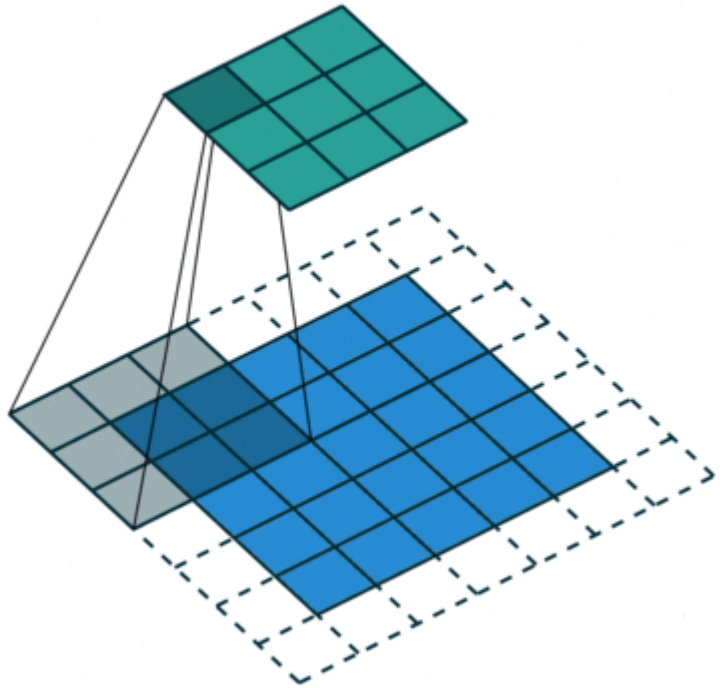
Stride 2



Feature Map

卷积层：步长

- **不同的步长与填充组合**
  - 步长为2，且有一圈填充

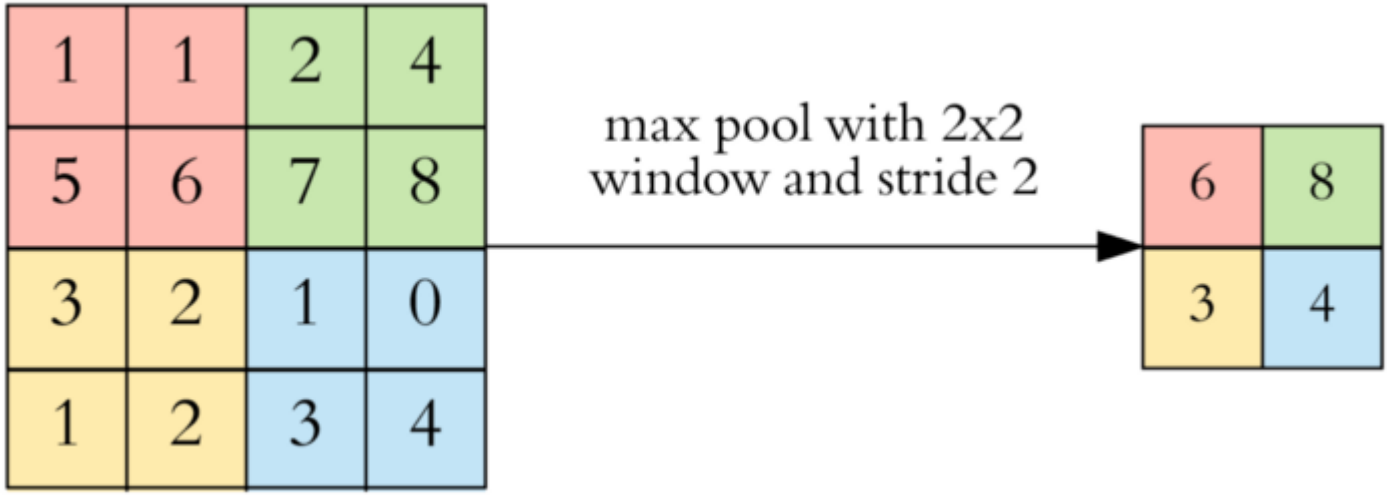


- 若输入图像大小为 $N \times N$,过滤器大小为 $F \times F$，步长为 S，填充了 P 圈，则输出图像的边长为

$$\frac{N+2P-F}{S}+1$$

池化层

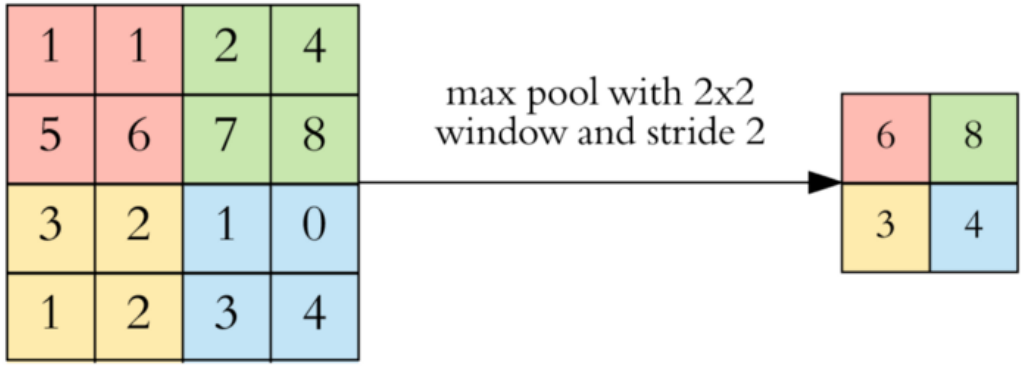
- 池化层可以降低数据尺寸
- 比较常见的，是用2*2的窗口，使用最大池化（max pooling），将图像大小缩小到上一层的1/4



池化层

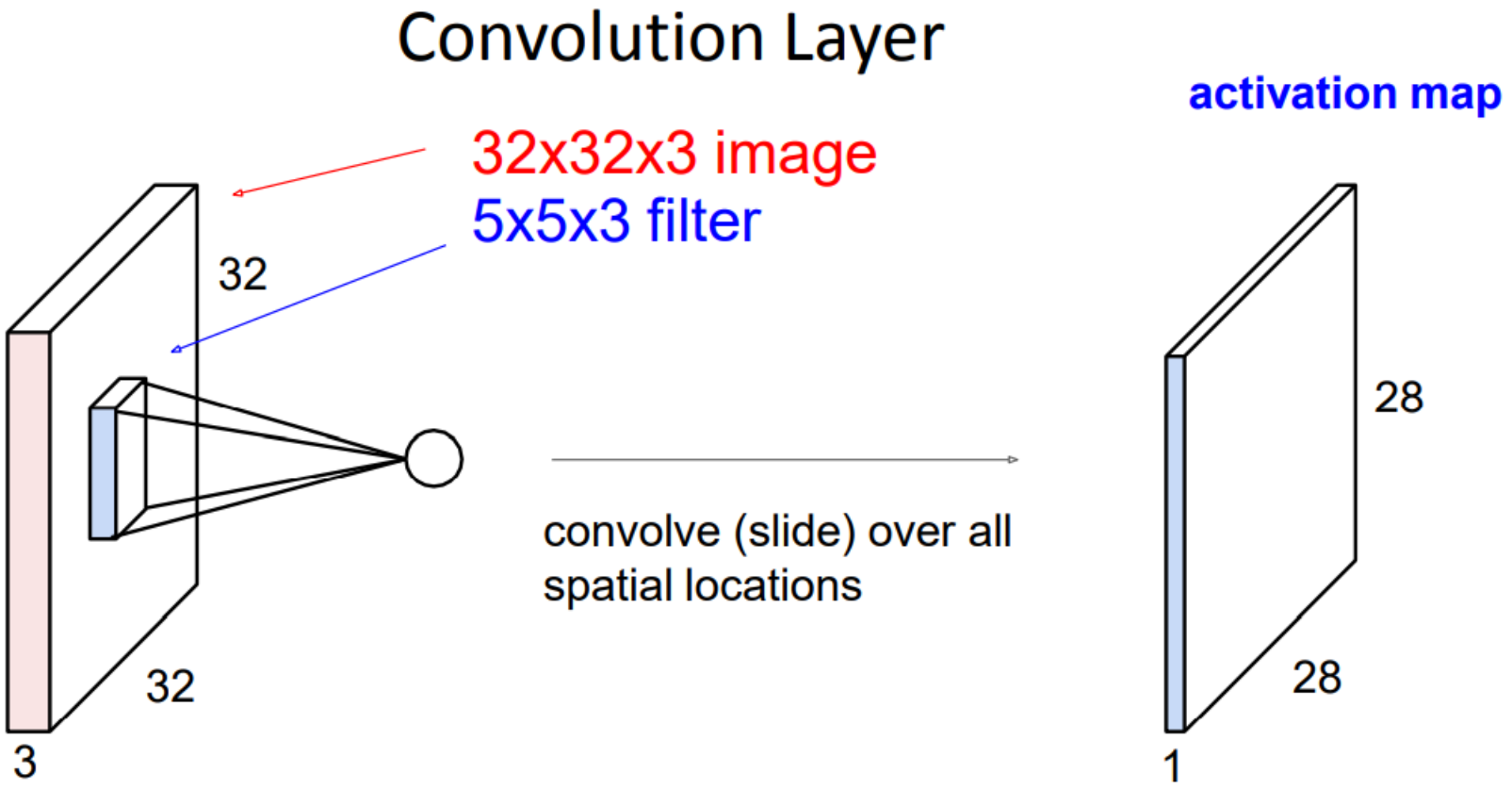
- **除了最大池化外，也有平均池化等其它方法；实践中一般认为，最大池化效果通常较好**

- **池化也可以使用不同的步长，或者加填充**
  - 常用的是2*2窗口、步长为2（即图中的情况）
  - 也可以用3*3窗口、步长为1、2或3



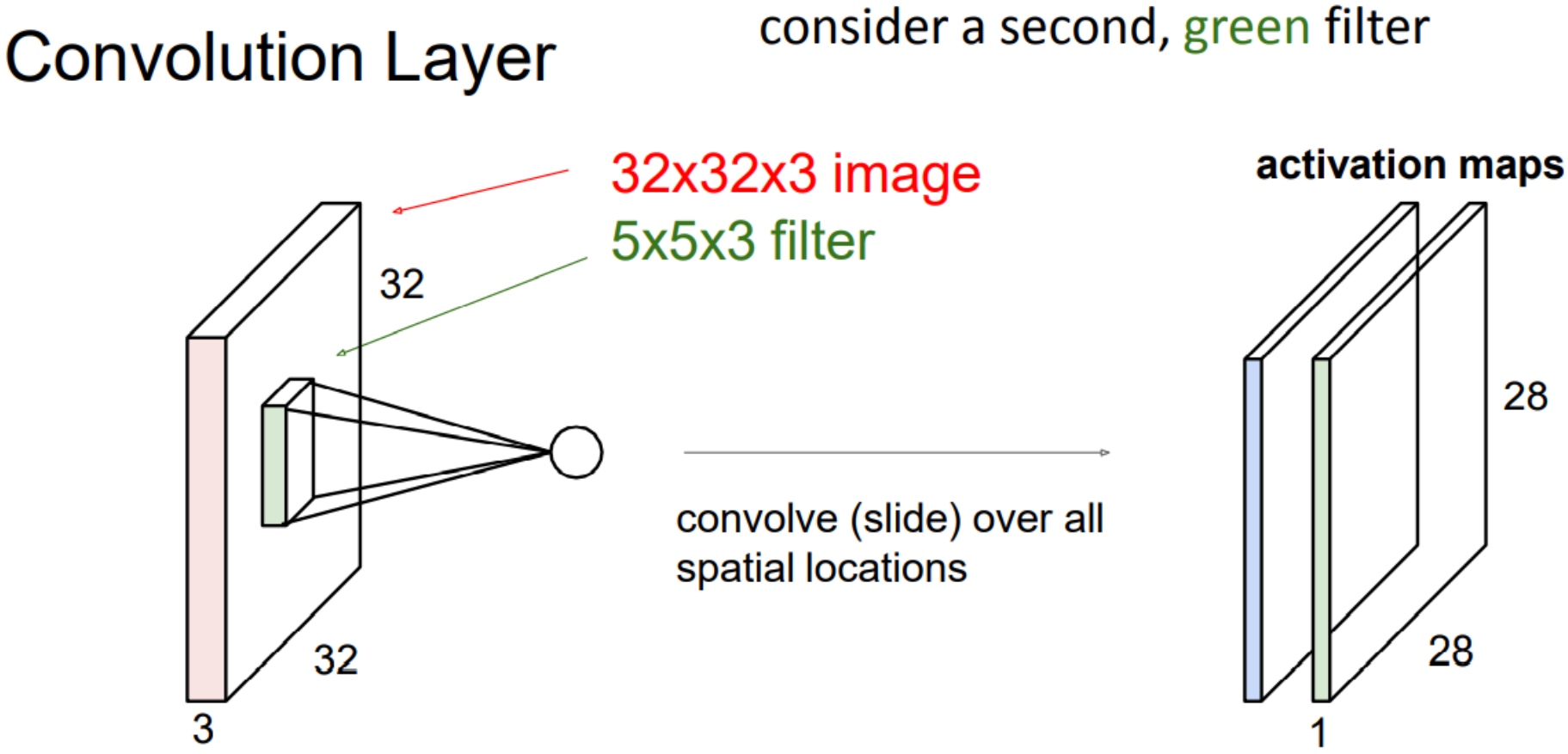
卷积神经网络：多重卷积

- 考虑32*32的RGB三通道彩色图像
- 使用5*5、步长为1的三通道过滤器，得到28*28



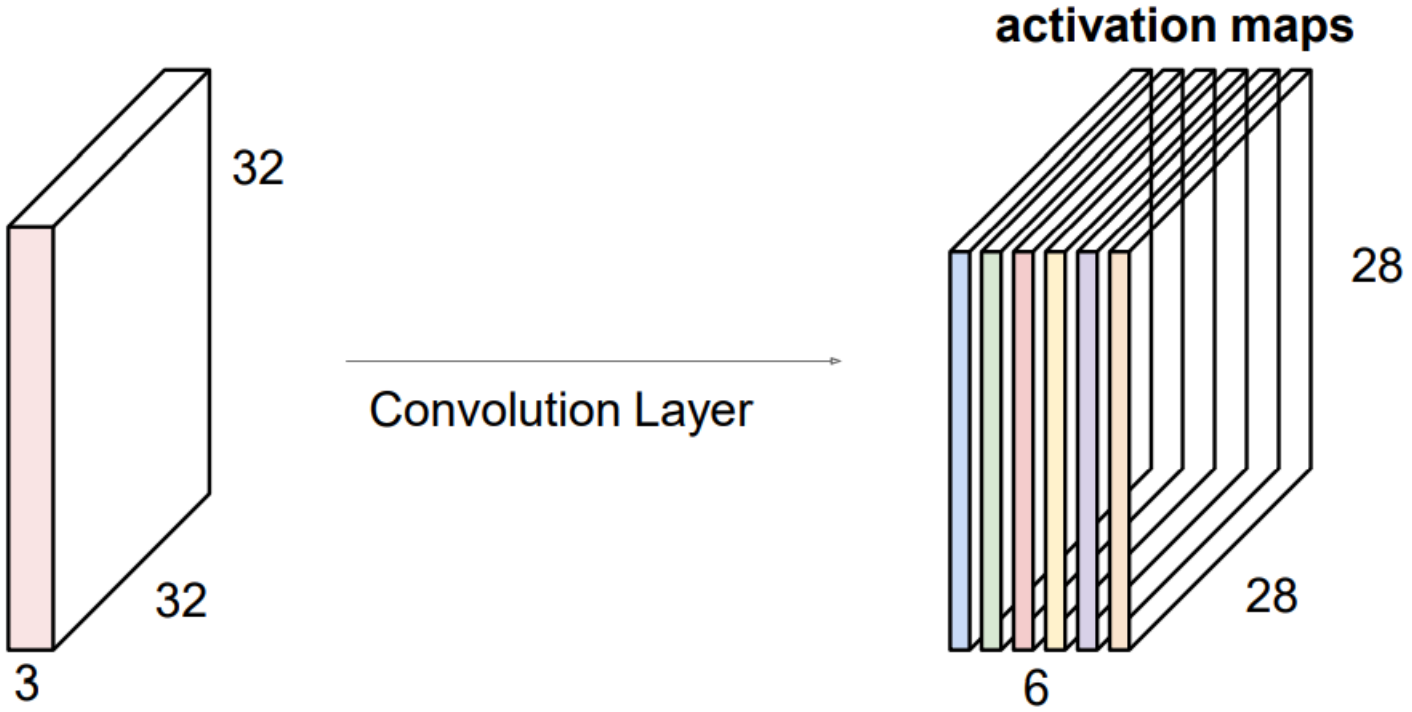
卷积神经网络：多重卷积

- 再使用一个不同的5*5、步长为1的三通道过滤器，得到另一个28*28



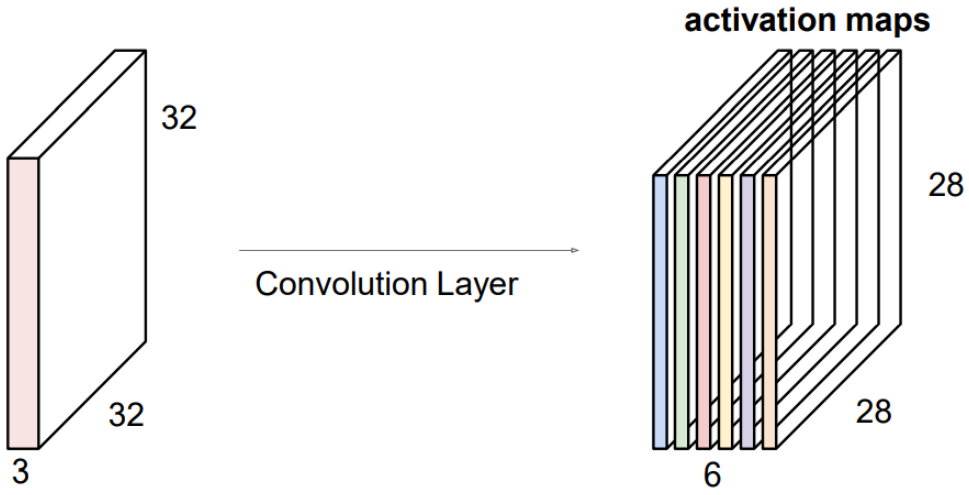
卷积神经网络：多重卷积

- 使用6个5*5的三通道过滤器，32*32*3的数据经过这样的卷积层，变成了28*28*6



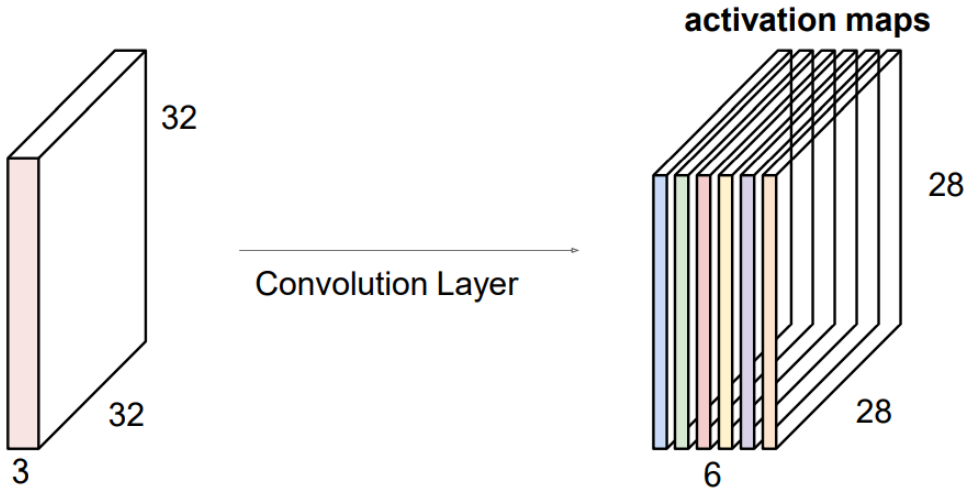
卷积神经网络：多重卷积

- 提问：目前为止的神经网络中有多少个参数?



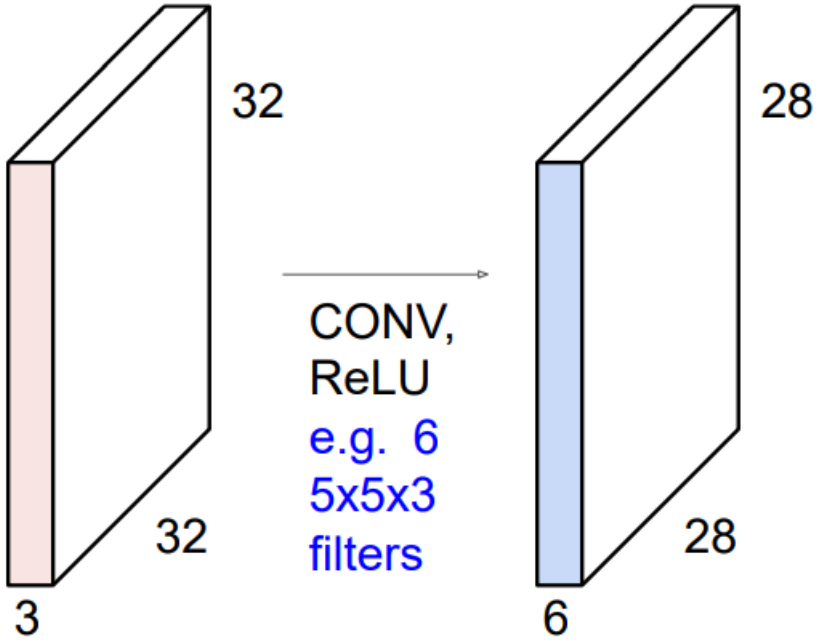
卷积神经网络：多重卷积

- **提问：目前为止的神经网络中有多少个参数?**
  - 每个单通道过滤器：5*5=25
  - 每个三通道过滤器及偏置：25*3+1=76
  - 六个这样的过滤器：76*6=456



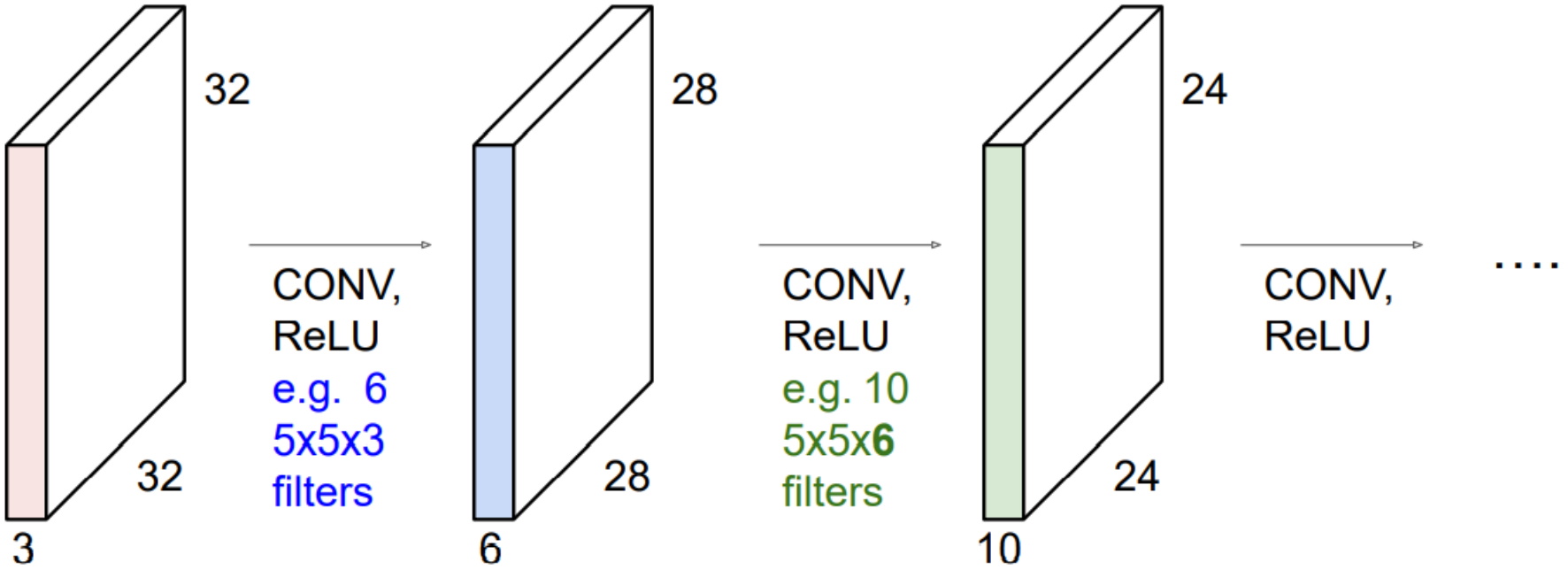
卷积神经网络：多重卷积

- 和我们之前讲过的神经网络一样，我们将卷积后 28*28*6 的结果通过一个激活层（一般用 ReLU），得到同样大小的输出（图中写作 “CONV，ReLU”）



卷积神经网络：多重卷积

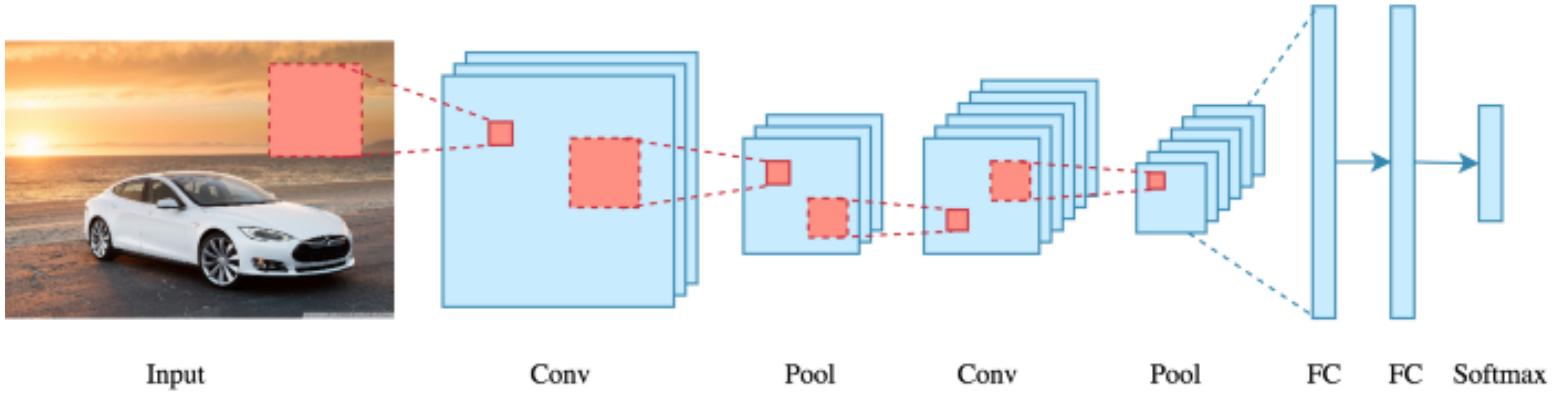
- 我们可以假如更多的隐藏层……



卷积神经网络：例子

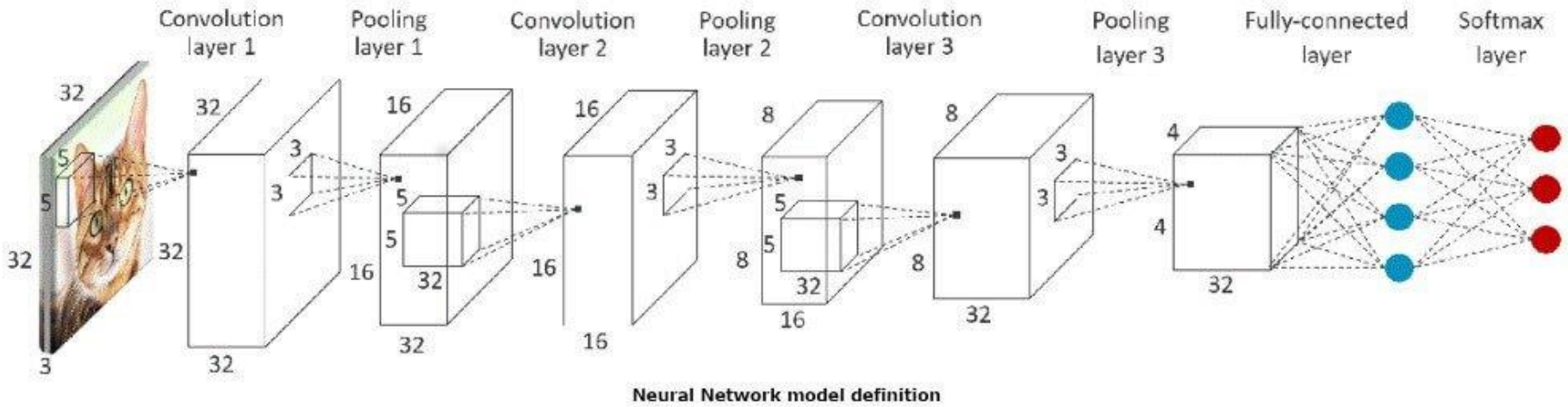
- 一般情况下，经过一些卷积层和池化层之后，可以采用全连接层，得到输出

- 如果输出是图像分类的概率，可以考虑最后一层使用 softmax 层



卷积神经网络：例子

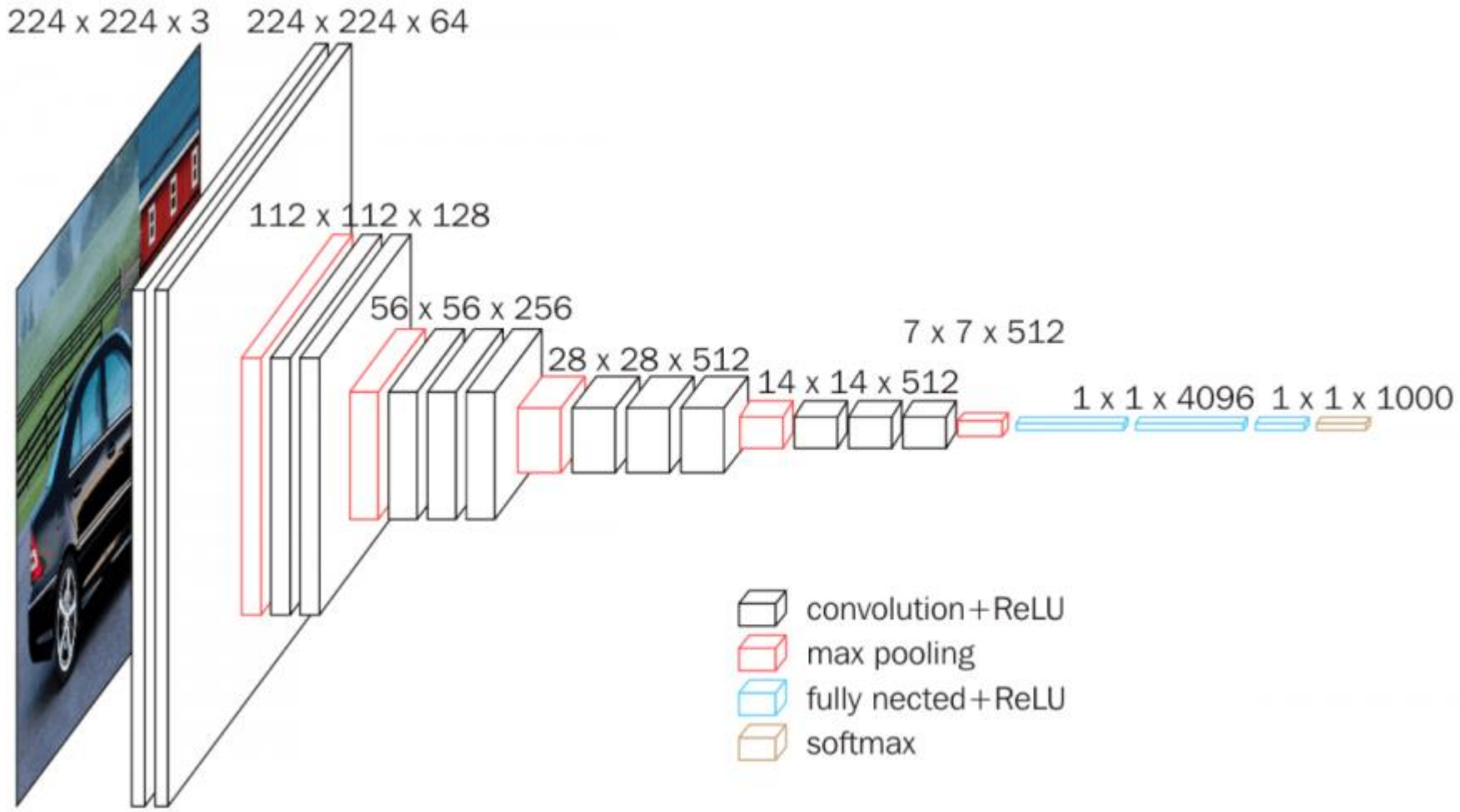
- 可以看懂以下卷积神经网络的结构吗



Neural Network model definition

卷积神经网络：VGG-16

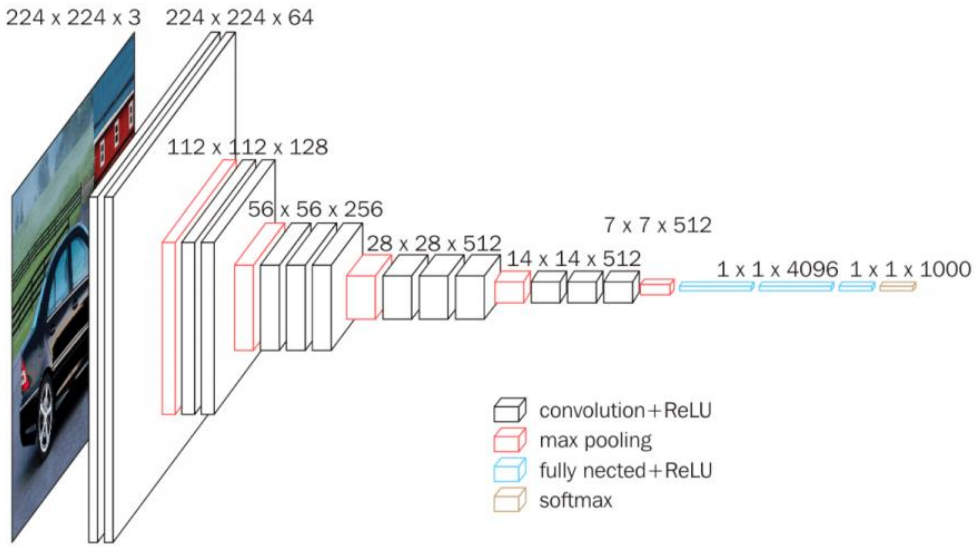
- 可以看懂以下卷积神经网络的结构吗



卷积神经网络：VGG-16

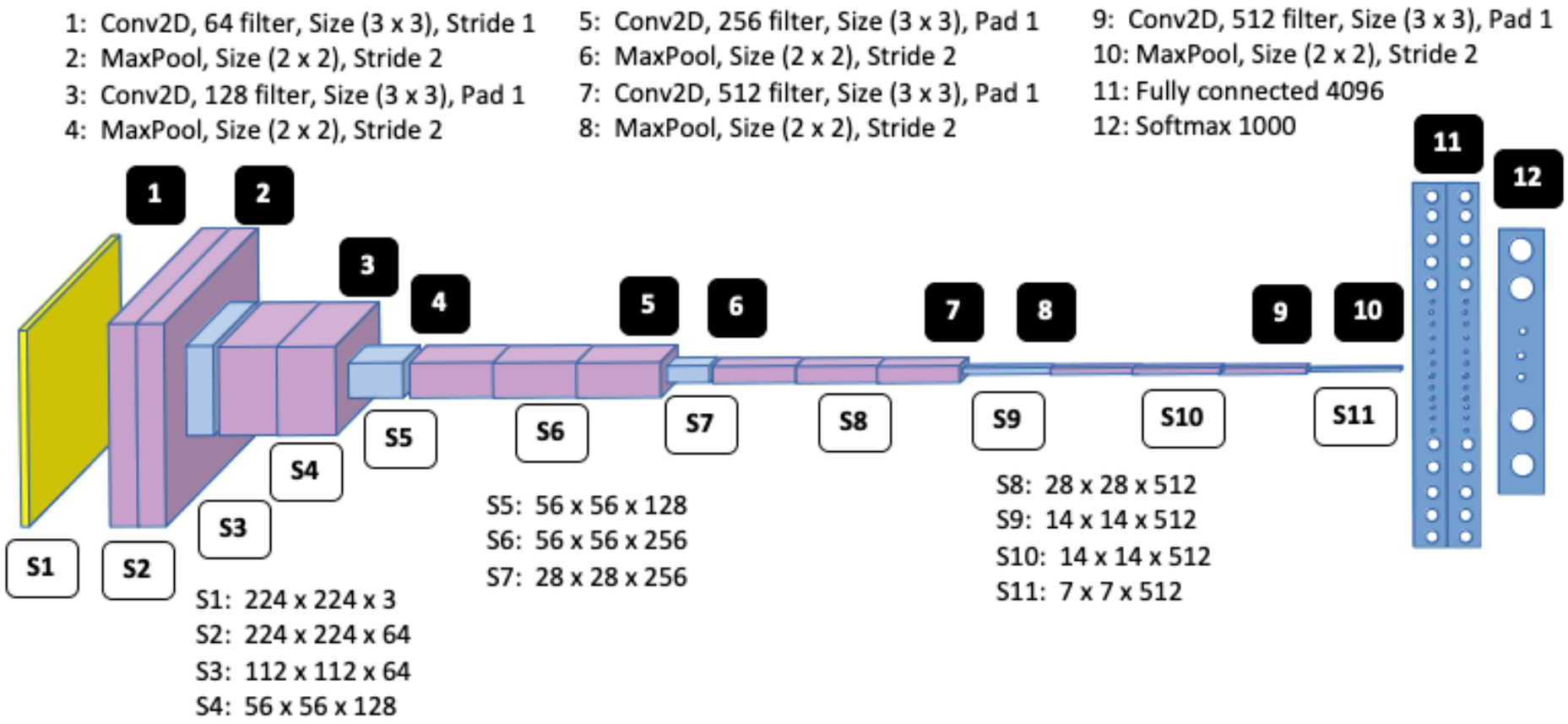
A stack of multiple (usually 1, 2, or 3) convolution layers of filter size 3 x 3, stride one, and padding 1, followed by a max-pooling layer of size 2 x 2, is the basic building block for all of these configurations. Different configurations of this stack were repeated in the network configurations to achieve different depths. The number associated with each of the configurations is the number of layers with weight parameters in them.

The convolution stacks are followed by three fully connected layers, two with size 4,096 and the last one with size 1,000. The last one is the output layer with Softmax activation. The size of 1,000 refers to the total number of possible classes in ImageNet.



ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

卷积神经网络：VGG-16



在 PyTorch 中使用卷积神经网络

CONV2D

CLASS torch.nn.Conv2d(*in_channels*, *out_channels*, *kernel_size*, *stride=1*, *padding=0*, *dilation=1*, *groups=1*, *bias=True*, *padding_mode='zeros'*, *device=None*, *dtype=None*) [SOURCE]

Applies a 2D convolution over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C_{in}, H, W) and output $(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ can be precisely described as:

$$\text{out}(N_i, C_{\text{out}_j}) = \text{bias}(C_{\text{out}_j}) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(C_{\text{out}_j}, k) \star \text{input}(N_i, k)$$

where $\star$ is the valid 2D cross-correlation operator, N is a batch size, C denotes a number of channels, H is a height of input planes in pixels, and W is width in pixels.

This module supports TensorFloat32.

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

- `stride` controls the stride for the cross-correlation, a single number or a tuple.
- `padding` controls the amount of padding applied to the input. It can be either a string {'valid', 'same'} or a tuple of ints giving the amount of implicit padding applied on both sides.

在 PyTorch 中使用卷积神经网络

```
class CNeuralNet(nn.Module):
    def __init__(self):
        super(NeuralNet, self).__init__()
        self.conv1 = nn.Sequential(
            nn.Conv2d(
                in_channels=1,
                out_channels=16,
                kernel_size=5,
                stride=1,
                padding=2,
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2),
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(
                in_channels=16,
                out_channels=32,
                kernel_size=5,
                stride=1,
                padding=2,
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size = 2),
        )
        # fully connected layer, output 10 classes
        self.out = nn.Linear(32 * 7 * 7, 10)
```

在 PyTorch 中使用卷积神经网络（续上页）

```
    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        # flatten the output of conv2 to (batch_size, 32 * 7 * 7)
        x = x.view(x.size(0), -1)
        output = self.out(x)
        return output, x    # return x for visualization

model = CNeuralNet().to(device)
```

练习题

在卷积神经网络中，对 32×32 的灰度图像使用 3×3、步长为 1、没有填充的过滤器，输出的数据为_____

A. 32×32

B. 30×30

C. 29×29

D. 64×64

练习题

在卷积神经网络中，对 32×32 的灰度图像使用 3×3、步长为 1、没有填充的过滤器，输出的数据为_____

A. 32×32

B. 30×30

C. 29×29

D. 64×64

【答案】：B

【解析】：卷积输出尺寸公式为-> 输出= (W输入尺寸 - F卷积核大小 + 2P无填充) / S步长 + 1 。

代入计算后尺寸为30

尝试理解卷积神经网络

卷积神经网络如何工作

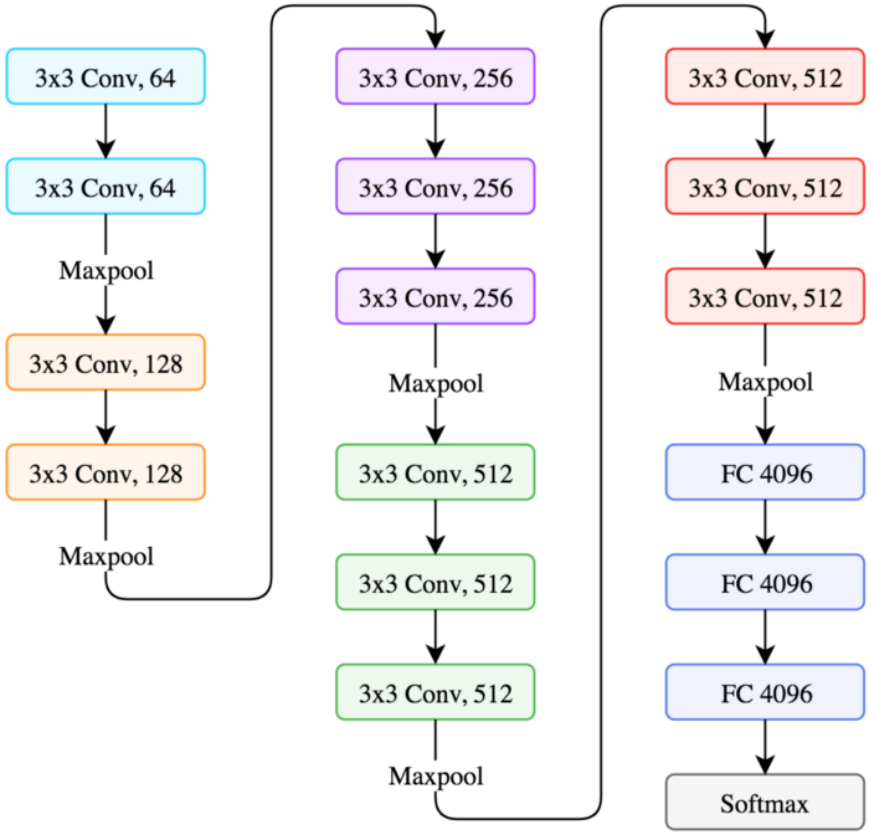
- **我们通过可视化，来尝试理解卷积神经网络是如何工作的**

- **我们将进行两类可视化**
  - 卷积结果可视化：使用训练好的神经网络，在其推理时，对中间层通道传递的数据进行可视化

  - 过滤器可视化（间接）：对训练好的卷积神经网络中的各个过滤器，我们寻找最能激活该过滤器的图案样式，对这些图案样式进行可视化

卷积神经网络如何工作

- 使用如下神经网络结构（VGG-13），在图像分类任务中训练

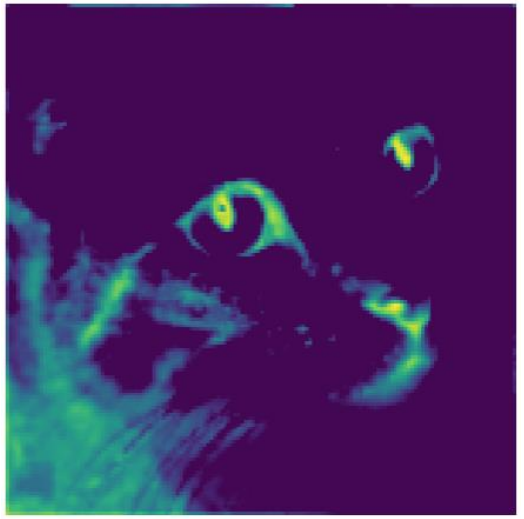


Applied Deep Learning - Part 4: Convolutional Neural Networks | by Arden Dertat | Towards Data Science

卷积神经网络如何工作：卷积结果可视化

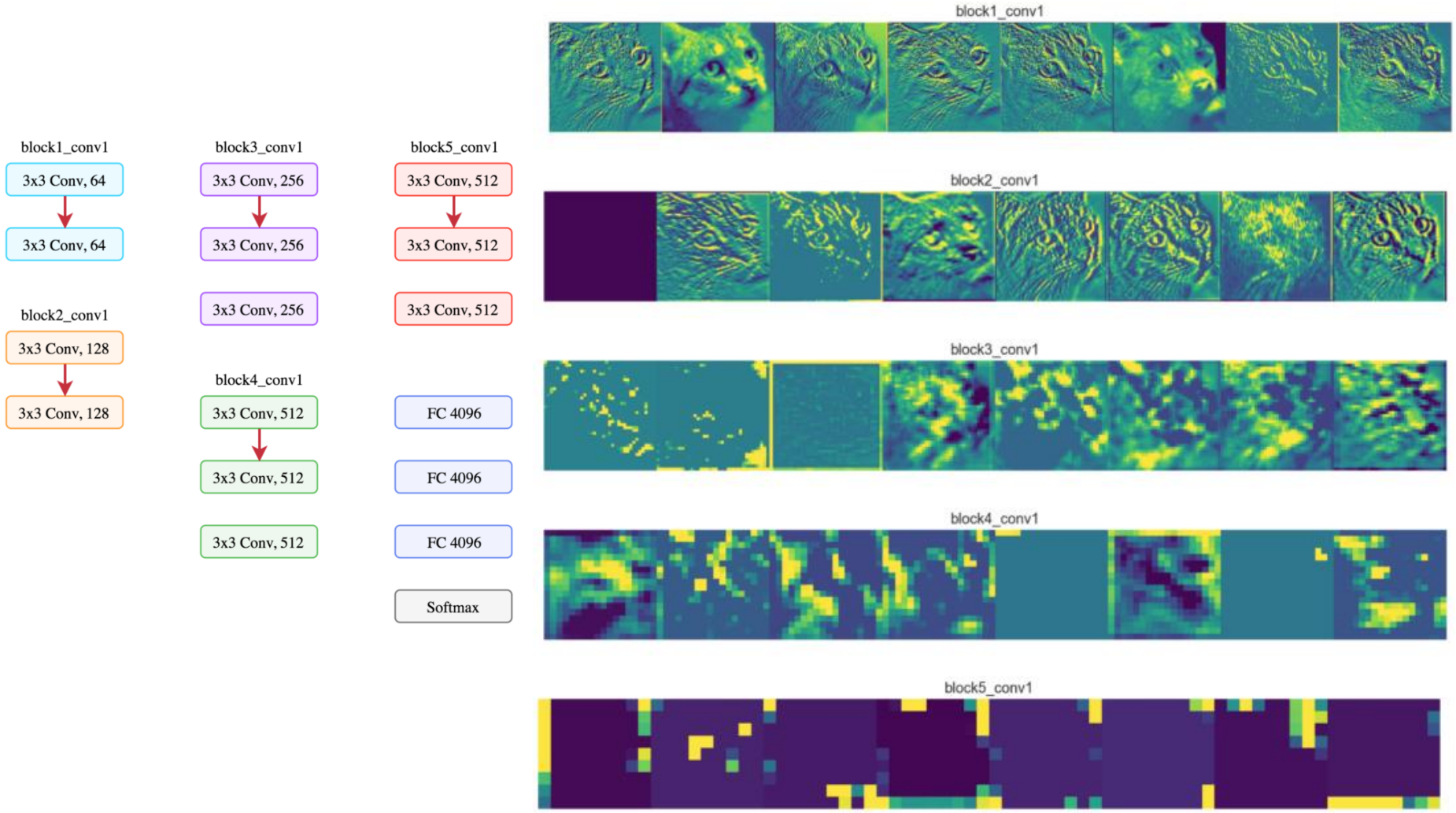
- **使用训练好的网络进行推理**
  - 输入为左图
- **对第一层（block1_conv1）得到的一个通道进行可视化，得到右图**
  - 分析：这个通道对图中眼睛和嘴的部分进行了强化/激活





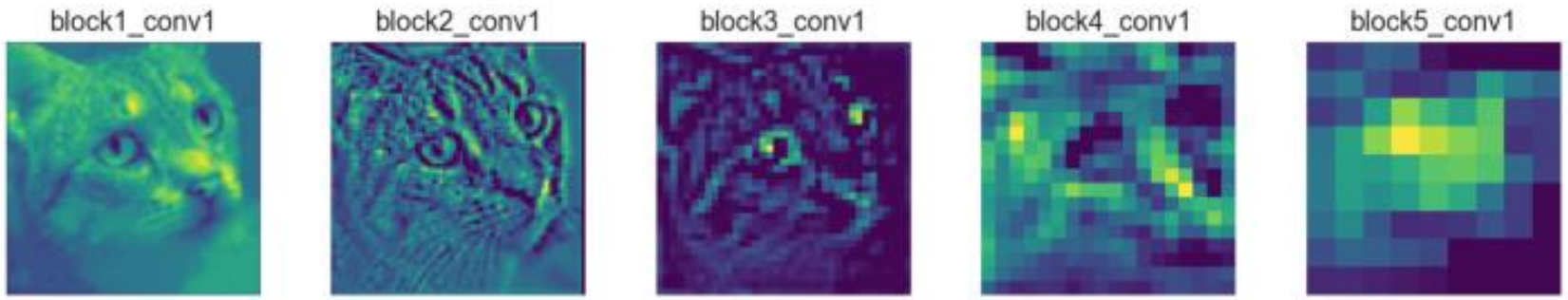
卷积神经网络如何工作：卷积结果可视化

- 截取下图中箭头处的部分通道，进行可视化



卷积神经网络如何工作：卷积结果可视化

- **从每一层中各选一张图，可以看得更加明显**
  - 卷积神经网络前几层保留了较多的原始图像信息，越往后，越抽象
  - 抽象出的信息，会与原始图像中的一些较高层次的概念相关，如本页图中，block5_conv1激活的部分可能就与猫的鼻子或眼睛有关系
  - 层数越深，体现的特征越稀疏，即被激活的特征少（参见上一页最后一行）



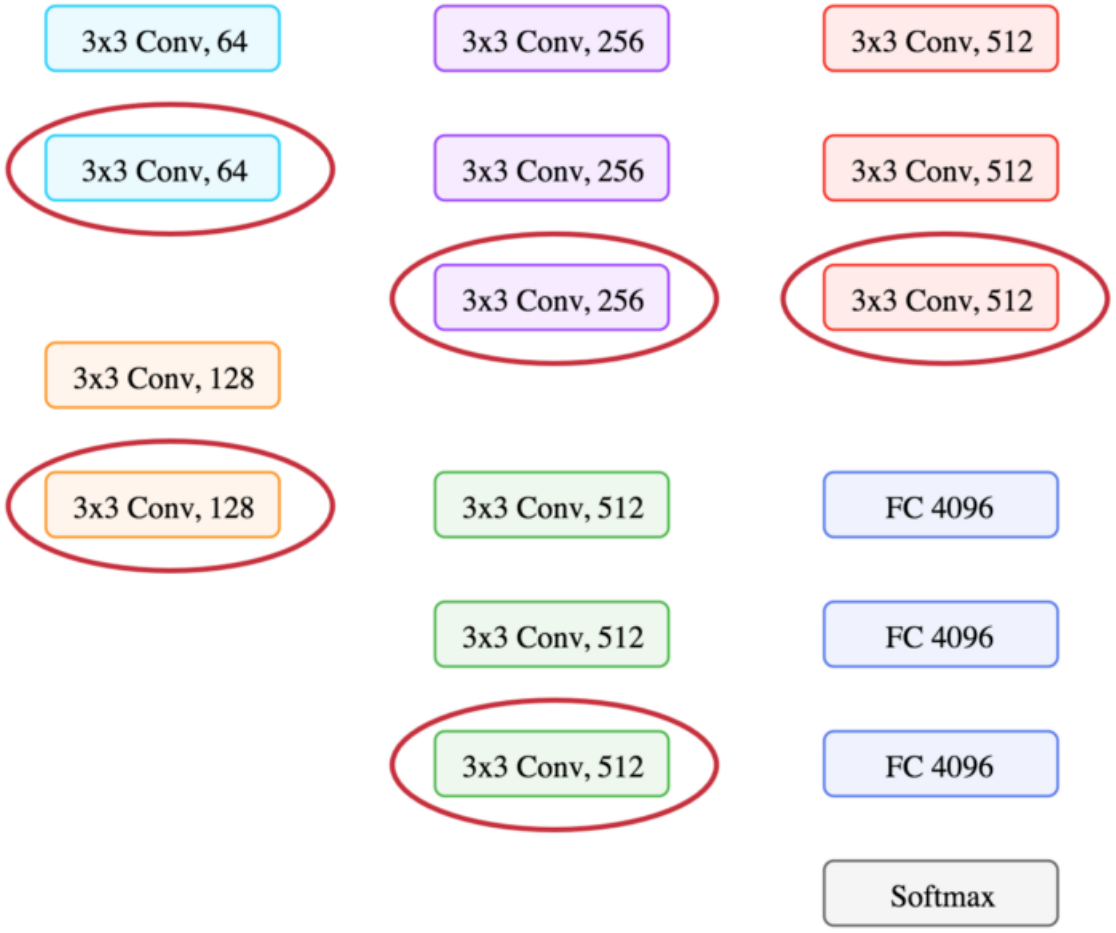
卷积神经网络如何工作：过滤器（间接）可视化

- **过滤器是3*3*3的矩阵加上偏置，直接可视化难以看出与图像相关的特征**

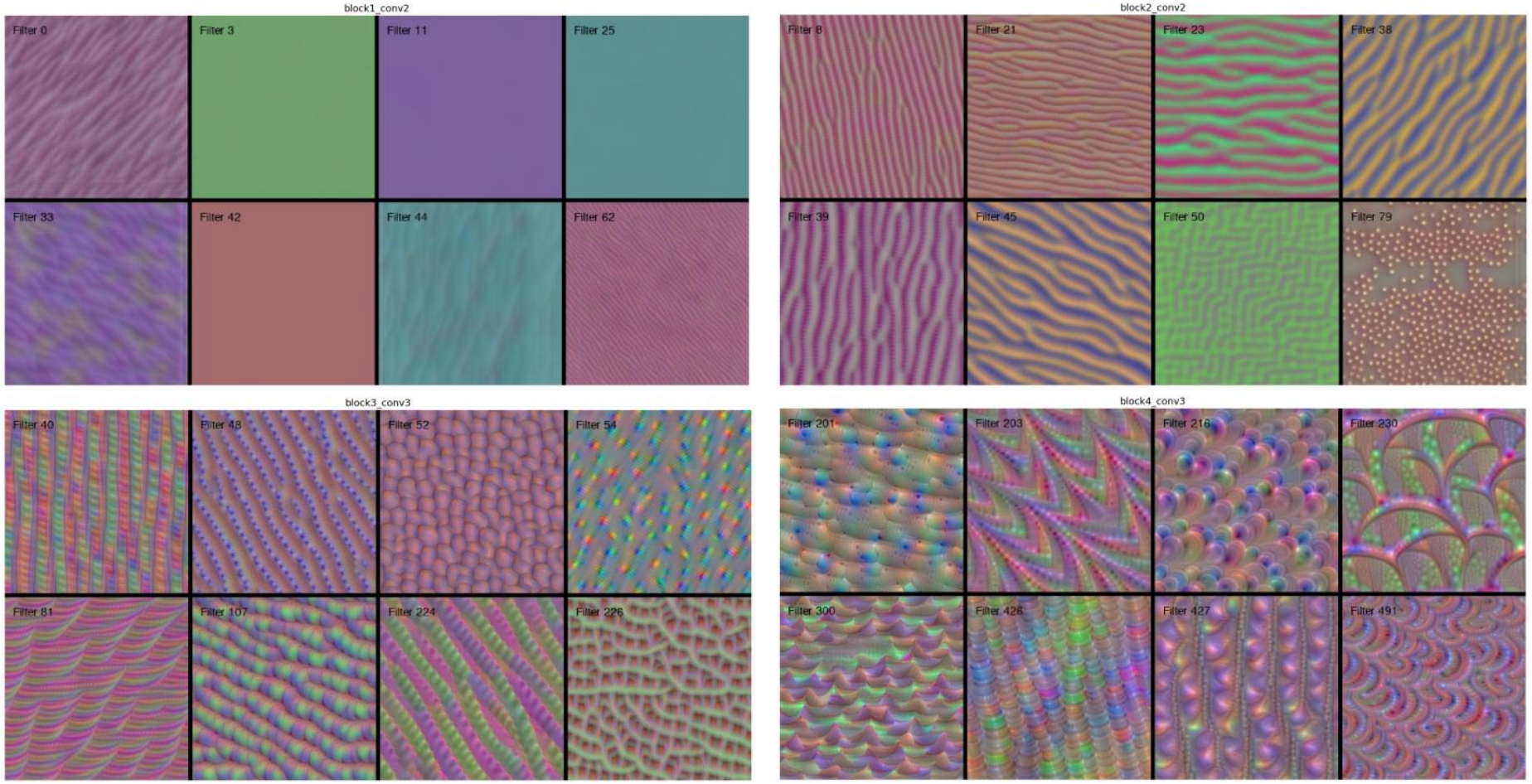
- **我们稍后对过滤器的可视化使用了一种特殊的方法，可视化的结果可以认为是“最容易让该过滤器激活的图像”**
  - 注意：以下两页的图不是过滤器本身，而是最能激活该过滤器的图像

卷积神经网络如何工作：过滤器（间接）可视化

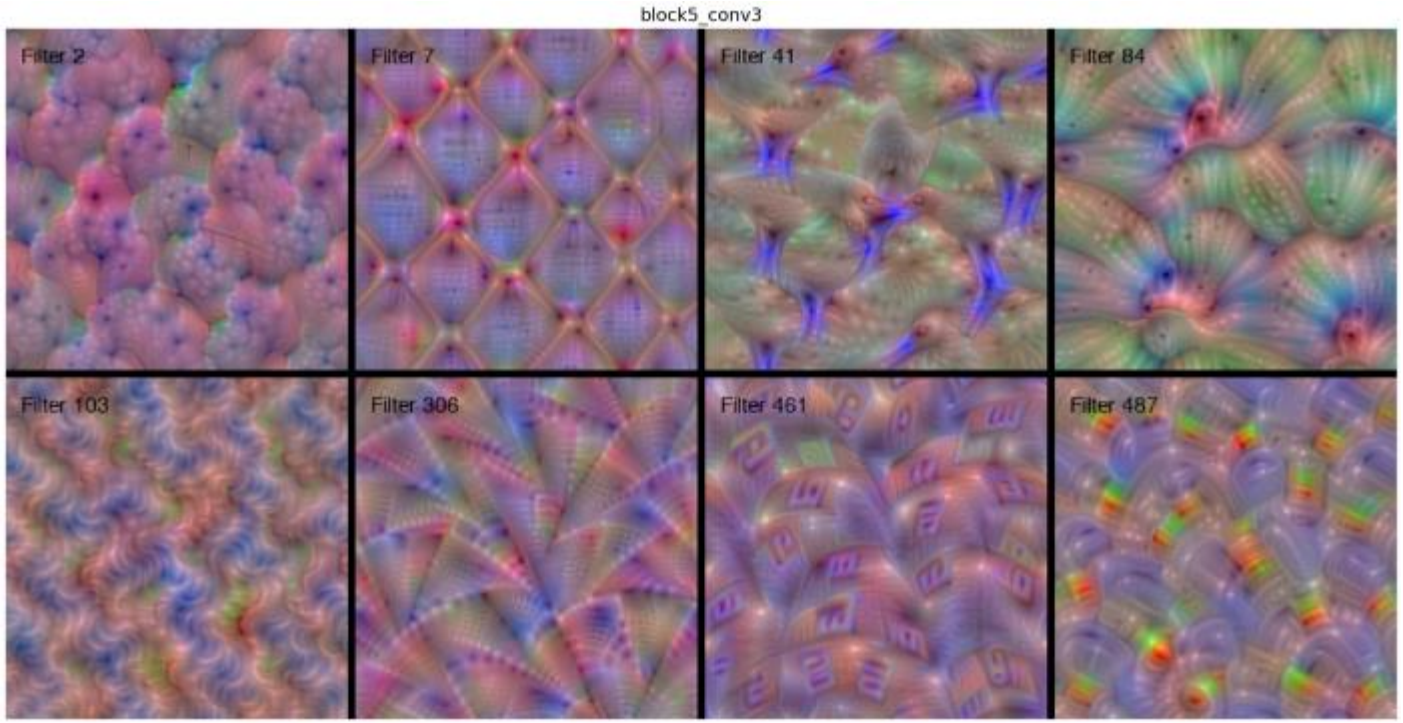
- 我们从如下位置中，选取一些过滤器，进行可视化



卷积神经网络如何工作：过滤器（间接）可视化

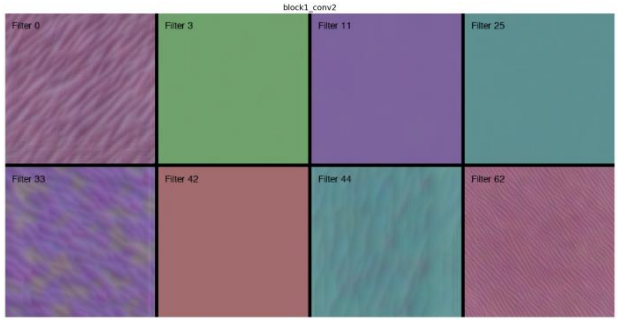


卷积神经网络如何工作：过滤器（间接）可视化

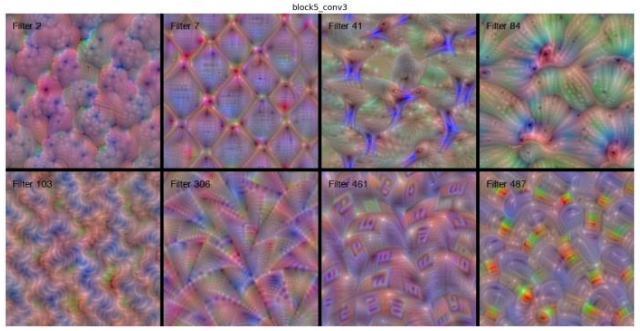


卷积神经网络如何工作：过滤器（间接）可视化

- **可以看出，前两层的过滤器，主要在检测颜色、边缘和简单的形状、纹理等**



- **深层中，可观察到重复的样式，这是因为在图像中，某些特征可能出现在任何位置**
  - 比如，狗的尾巴可能在图片下方，也可能在上方



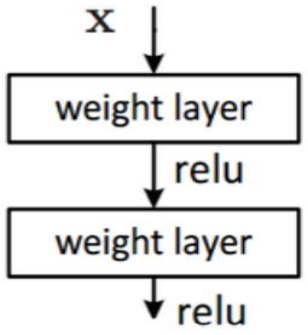
残差网络
(Residual Network)

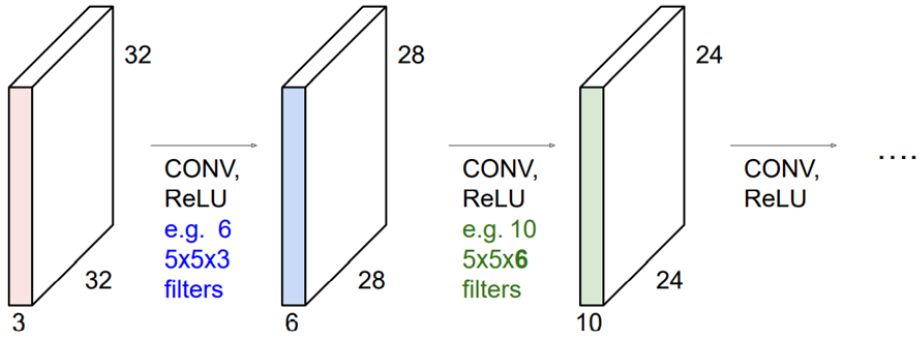
残差网络

- 神经网络的层数越多，其潜力越大；但层数多的深度神经网络训练起来通常很困难
- 此时，使用残差能有效提高深度网络的训练效果
- 近年来，在图像领域，残差网络（准确地说是残差卷积神经网络）比单纯的卷积神经网络效果好

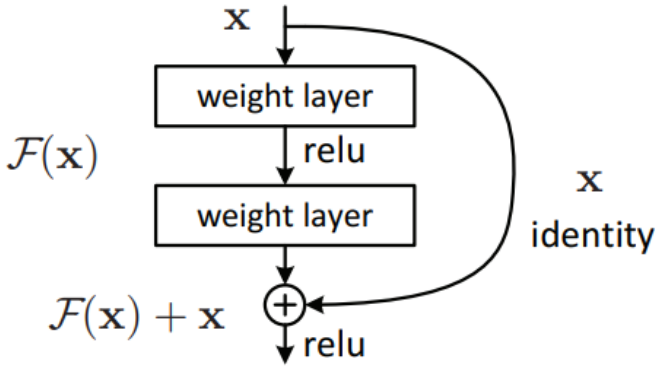
残差网络

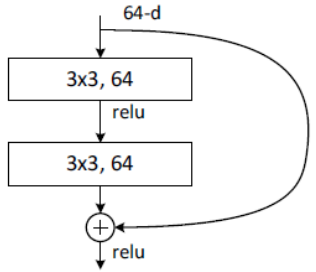
- 不使用残差的网络：

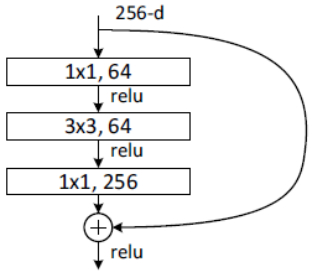




- 使用残差的网络：

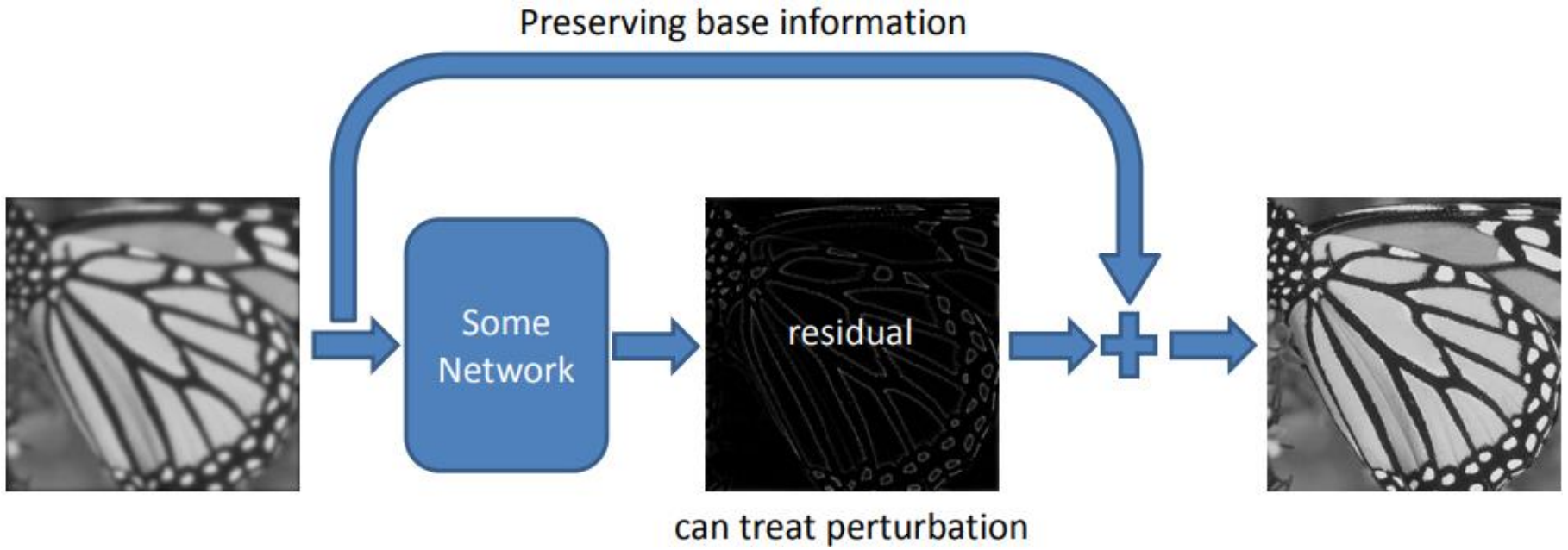




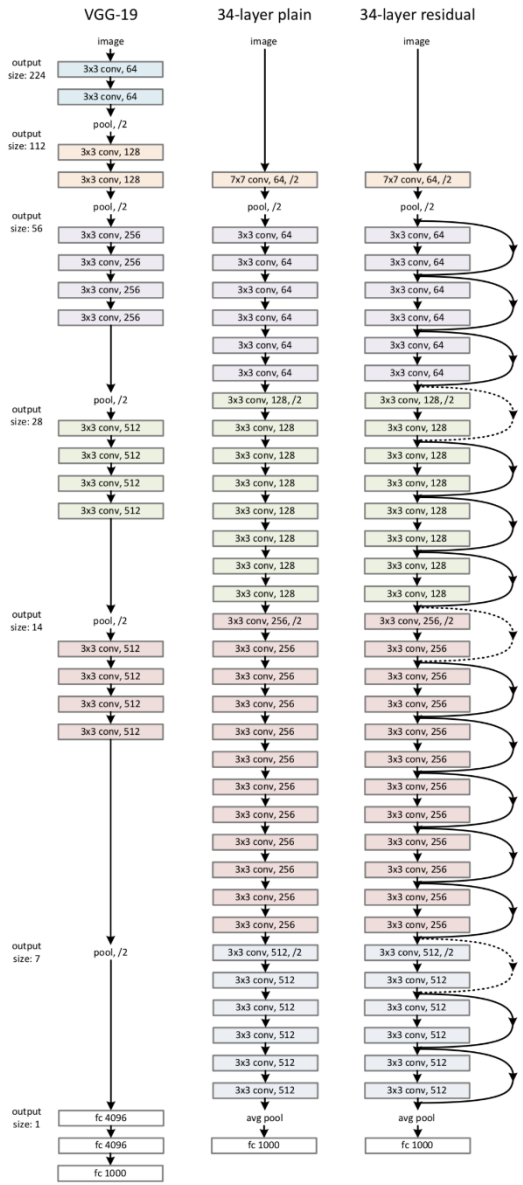


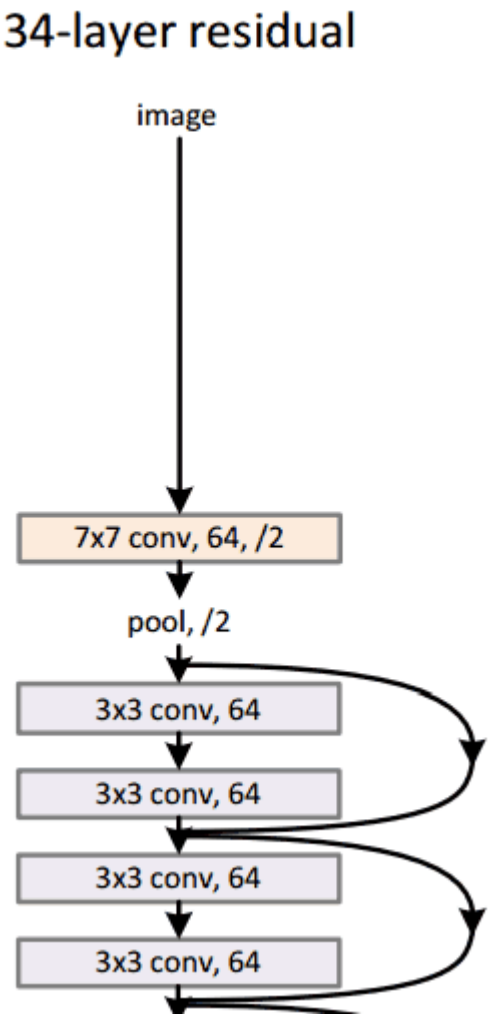
残差网络

- 残差与原图像叠加的效果



残差网络


VGG-19
34-layer plain
34-layer residual
image
output size: 224
3x3 conv, 64
pool, /2
output size: 112
3x3 conv, 128
output size: 56
3x3 conv, 256
output size: 28
3x3 conv, 512
output size: 14
output size: 7
output size: 1
fc 4096
fc 1000
7x7 conv, 64, /2
3x3 conv, 128, /2
3x3 conv, 256, /2
3x3 conv, 512, /2
avg pool


34-layer residual
image
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64

残差网络 ResNet

- **Pytorch 中有建好且预训练好的 ResNet 网络，可直接使用**

- [ResNet — Torchvision 0.14 documentation (pytorch.org)]

生成对抗网络
(Generative Adversarial Network, GAN)

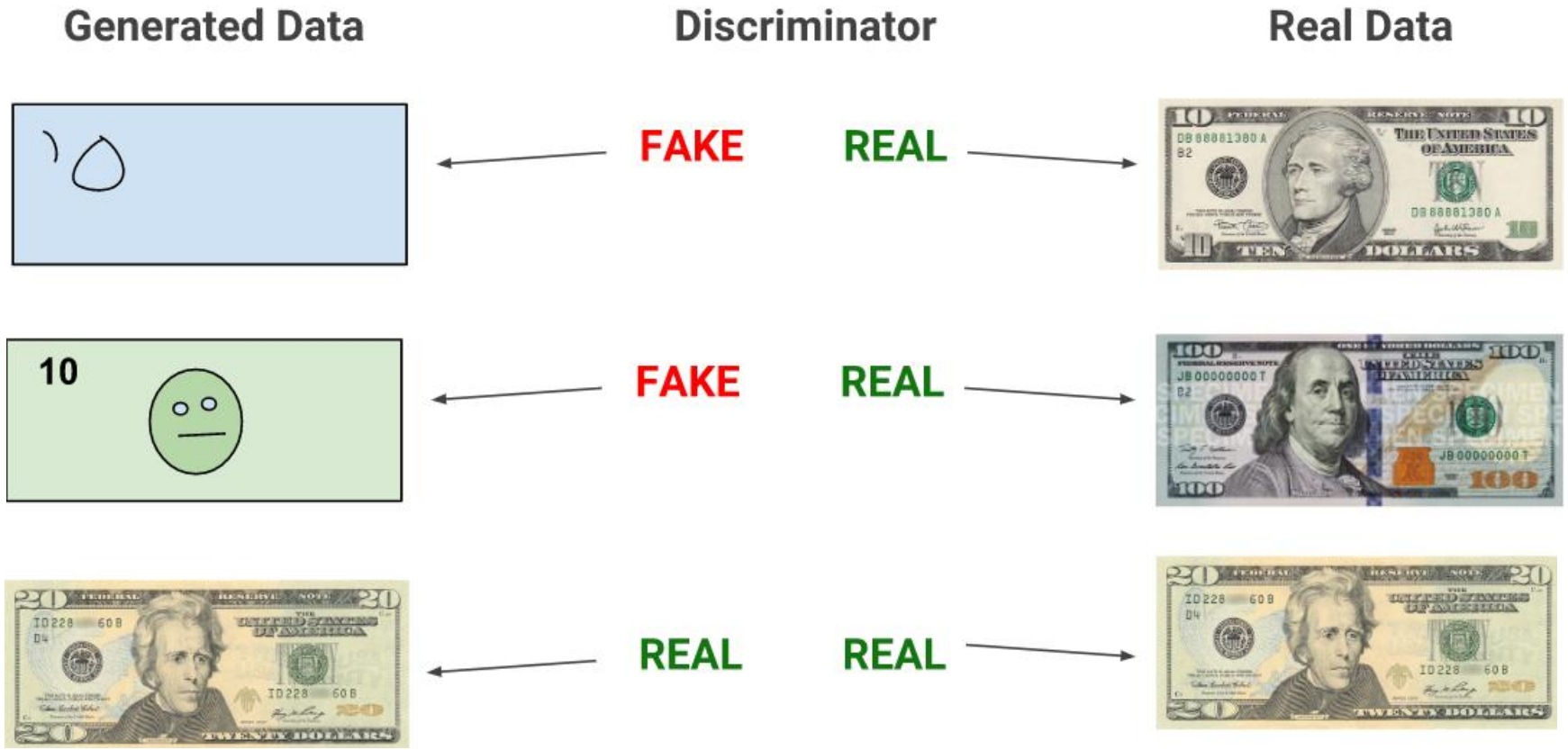
生成对抗网络

- **生成对抗网络包含两个部分的神经网络**
  - **生成（generative）模型**
    - 生成模型的任务是生成看起来真实的数据
  - **判别（discriminative）模型**
    - 判别模型的任务是判断数据是真实的还是生成的



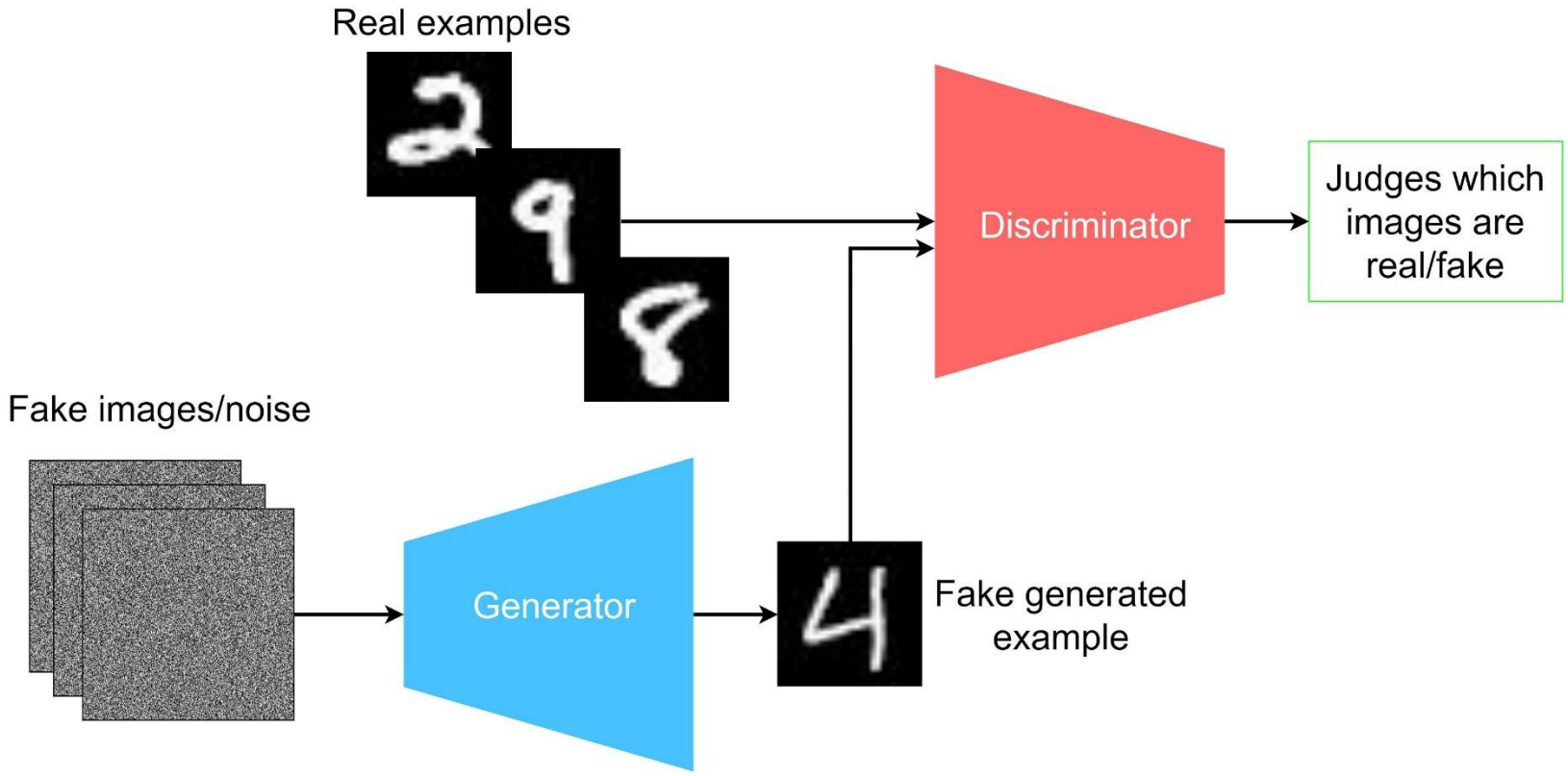
生成对抗网络

- 随着训练的进行，生成模型生成的模型越来越“逼真”，判别模型的判别水平通常也越来越高



生成对抗网络

- 生成对抗网络在图像识别或图像生成中都可得到应用



生成对抗网络的训练

- **在训练生成对抗网络时，可以交替对生成模型和判别模型进行训练**
  - 生成模型不变，训练判别模型
  - 判别模型不变，训练生成模型

- **训练中保持生成模型和判别模型“水平相当”，“共同进步”**

实践经验

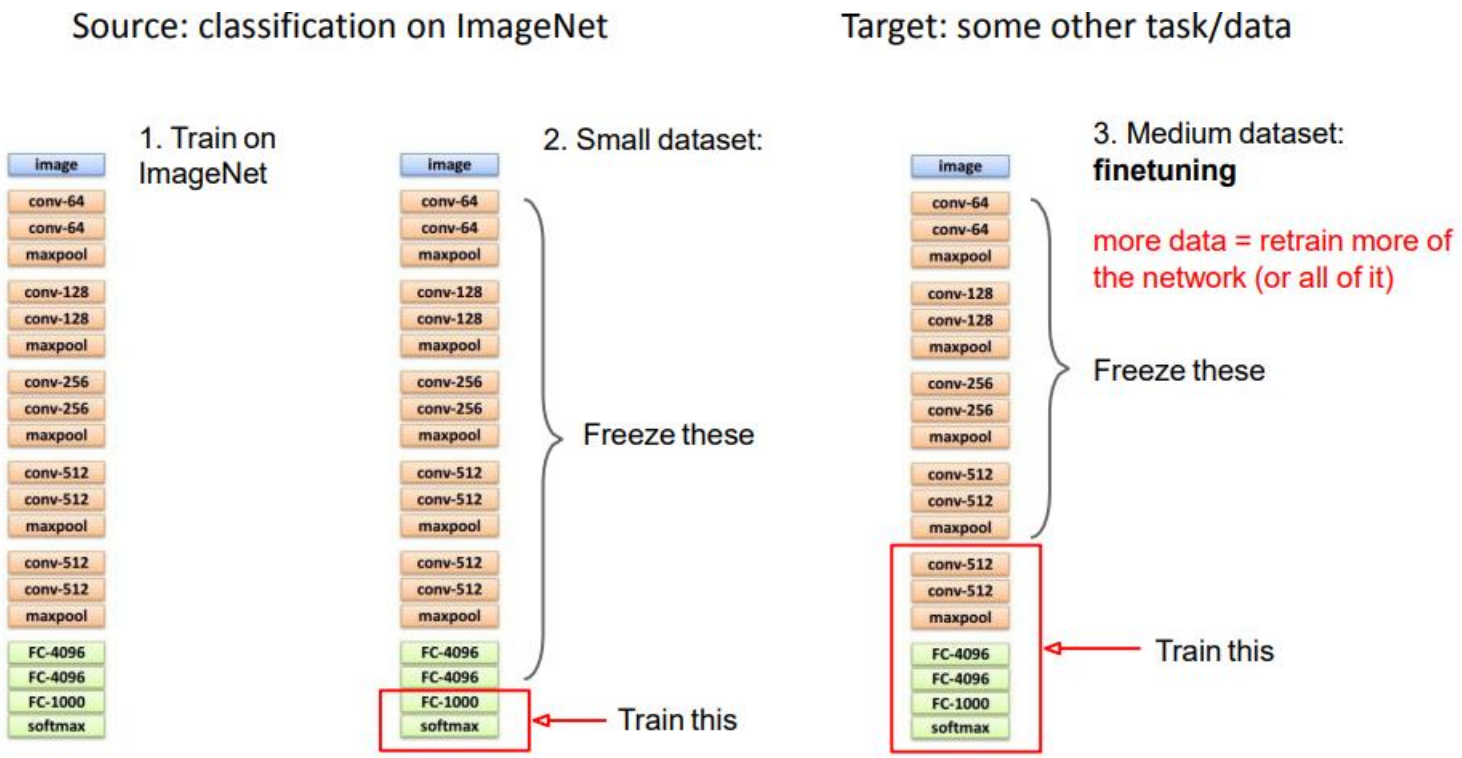
基本原则

- 使用 mini-batch
- 隐藏层使用 ReLU 激活函数
- 进行适当的权重参数初始化
- 使用 dropout 或 batch normalization
- 考虑使用正则化项

- 当训练样本数量不够多时，使用数据增强（通过翻转、拉伸、扭转、变色等生成更多样本)

迁移学习（transfer learning）

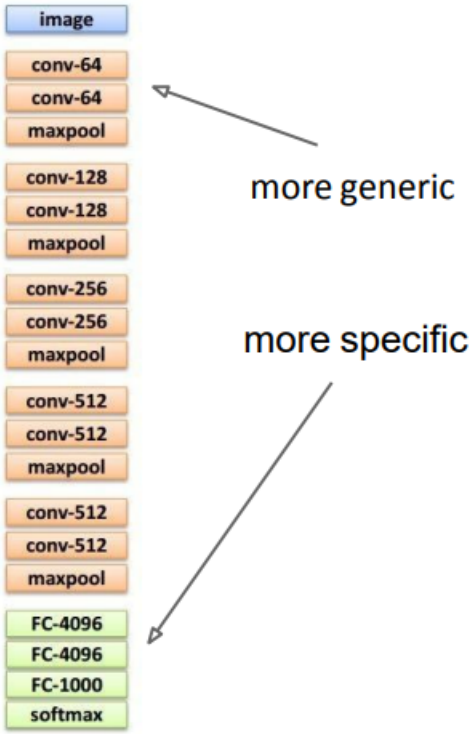
- **使用预训练好的网络参数作为骨干（backbone）网络，尤其是网络前端的卷积部分**



Another option: use network as feature extractor, train SVM on extracted features for target task

迁移学习

- **网络前端部分，主要是从图片中提取通用的特征；末端的部分，是利用这些特征进行分类等特殊任务**



	very similar dataset	very different dataset
very little data	Use linear classifier on top layer	You're in trouble… Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

迁移学习

- 首先，选一个在 ImageNet 上训练过的带参数的神经网络，如 VGG-16，ResNet 等，作为骨干网络
- 去掉末端的一些全连接层，把此前最后一层的网络输出看作“特征”
- 根据具体的机器学习任务以及样本数量、质量的情况，选择用支持向量机（SVM）或神经网络模型，进行训练
- 之后也可以在根据样本数量、质量的情况，考虑训练微调整个网络的参数

在 PyTorch 中使用骨干网络

- **torchvision 中可以直接下载训练好的骨干网络**
  - 包含了预设的网络结合，和预训练过的参数
  - [Models and pre-trained weights — Torchvision 0.14 documentation (pytorch.org)]

Initializing pre-trained models

As of v0.13, TorchVision offers a new Multi-weight support API for loading different weights to the existing model builder methods:

```
from torchvision.models import resnet50, ResNet50_Weights

# Old weights with accuracy 76.130%
resnet50(weights=ResNet50_Weights.IMAGENET1K_V1)

# New weights with accuracy 80.858%
resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)

# Best available weights (currently alias for IMAGENET1K_V2)
# Note that these weights may change across versions
resnet50(weights=ResNet50_Weights.DEFAULT)

# Strings are also supported
resnet50(weights="IMAGENET1K_V2")

# No weights - random initialization
resnet50(weights=None)
```

在 PyTorch 中使用骨干网络

- **我们可以基于骨干网络针对我们的问题设计新的网络（如修改该网络最后输出的维度；去掉该网络的最后几层，和我们设计的输出层连接）**
  - Finetuning Torchvision Models — PyTorch Tutorials 1.2.0 documentation
  - 用 print(my_model) 或 my_model._modules可以看到网络各层的名称（下图中的“fc”就是网络最后一层的名称）

Resnet was introduced in the paper Deep Residual Learning for Image Recognition. There are several variants of different sizes, including Resnet18, Resnet34, Resnet50, Resnet101, and Resnet152, all of which are available from torchvision models. Here we use Resnet18, as our dataset is small and only has two classes. When we print the model, we see that the last layer is a fully connected layer as shown below:

```
(fc): Linear(in_features=512, out_features=1000, bias=True)
```

Thus, we must reinitialize `model.fc` to be a Linear layer with 512 input features and 2 output features with:

```
model.fc = nn.Linear(512, num_classes)
```

在 PyTorch 中使用骨干网络

- **训练时，若我们希望骨干网络部分的参数不变化（即固定预训练好的网络，只训练其它的部分）**
  - 我们需要将预训练好的参数的 .requires_grad 设为 False
  - Finetuning Torchvision Models — PyTorch Tutorials 1.2.0 documentation

Set Model Parameters' .requires_grad attribute

This helper function sets the `.requires_grad` attribute of the parameters in the model to False when we are feature extracting. By default, when we load a pretrained model all of the parameters have `.requires_grad=True`, which is fine if we are training from scratch or finetuning. However, if we are feature extracting and only want to compute gradients for the newly initialized layer then we want all of the other parameters to not require gradients. This will make more sense later.

```
def set_parameter_requires_grad(model, feature_extracting):
    if feature_extracting:
        for param in model.parameters():
            param.requires_grad = False
```

```
if model_name == "resnet":
    """ Resnet18
    """
    model_ft = models.resnet18(pretrained=use_pretrained)
    set_parameter_requires_grad(model_ft, feature_extract)
    num_ftrs = model_ft.fc.in_features
    model_ft.fc = nn.Linear(num_ftrs, num_classes)
    input_size = 224
```